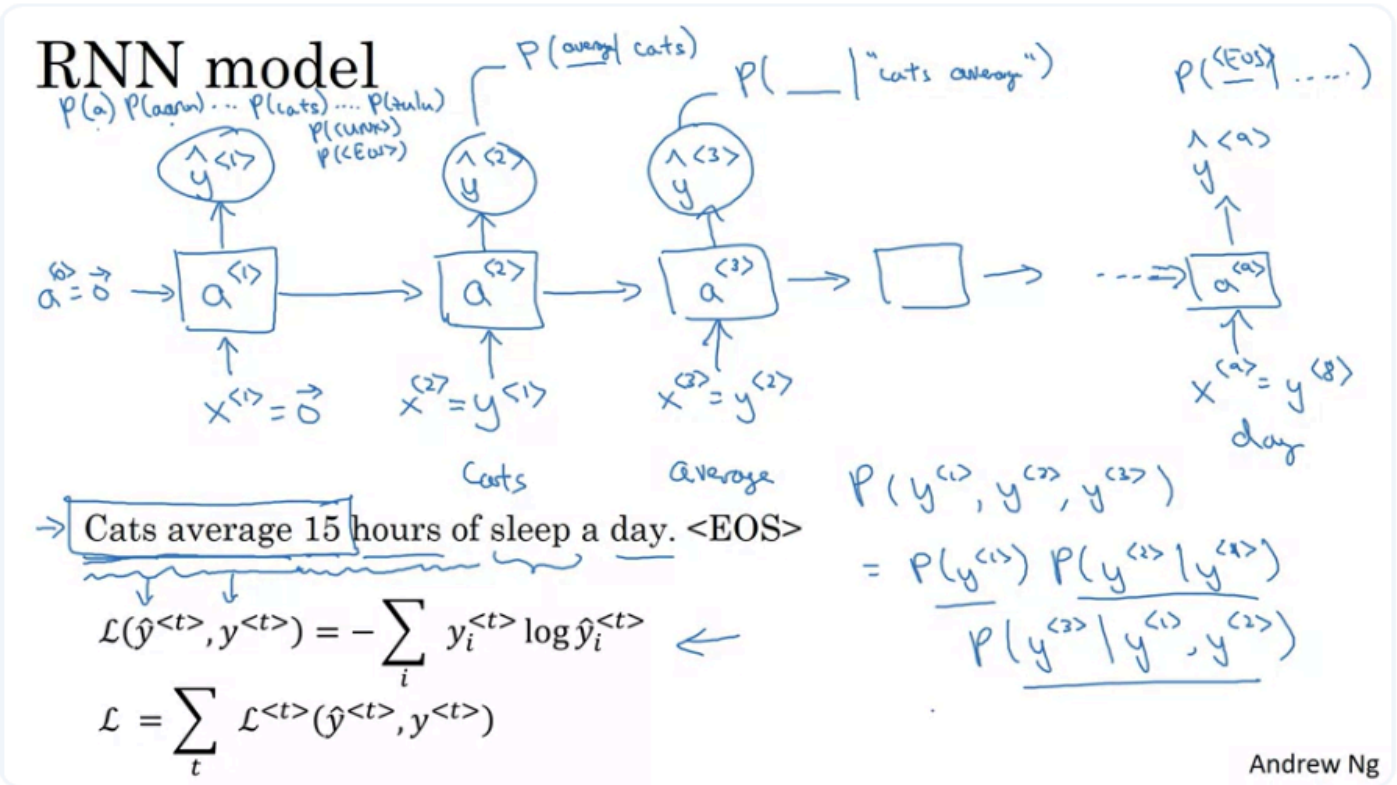
Recurrent Neural Networks Part 2

Language Model and Sequence Generation

is the model trying to predict the sentences (or words) based on statistics



RNN Language Models

What Is Language Modeling?

Language modeling is a fundamental task in Natural Language Processing.

A **language model** estimates the probability of a sequence of words. In other words, it answers the question:

> How likely is this sentence to appear in natural language?

Given a sentence:

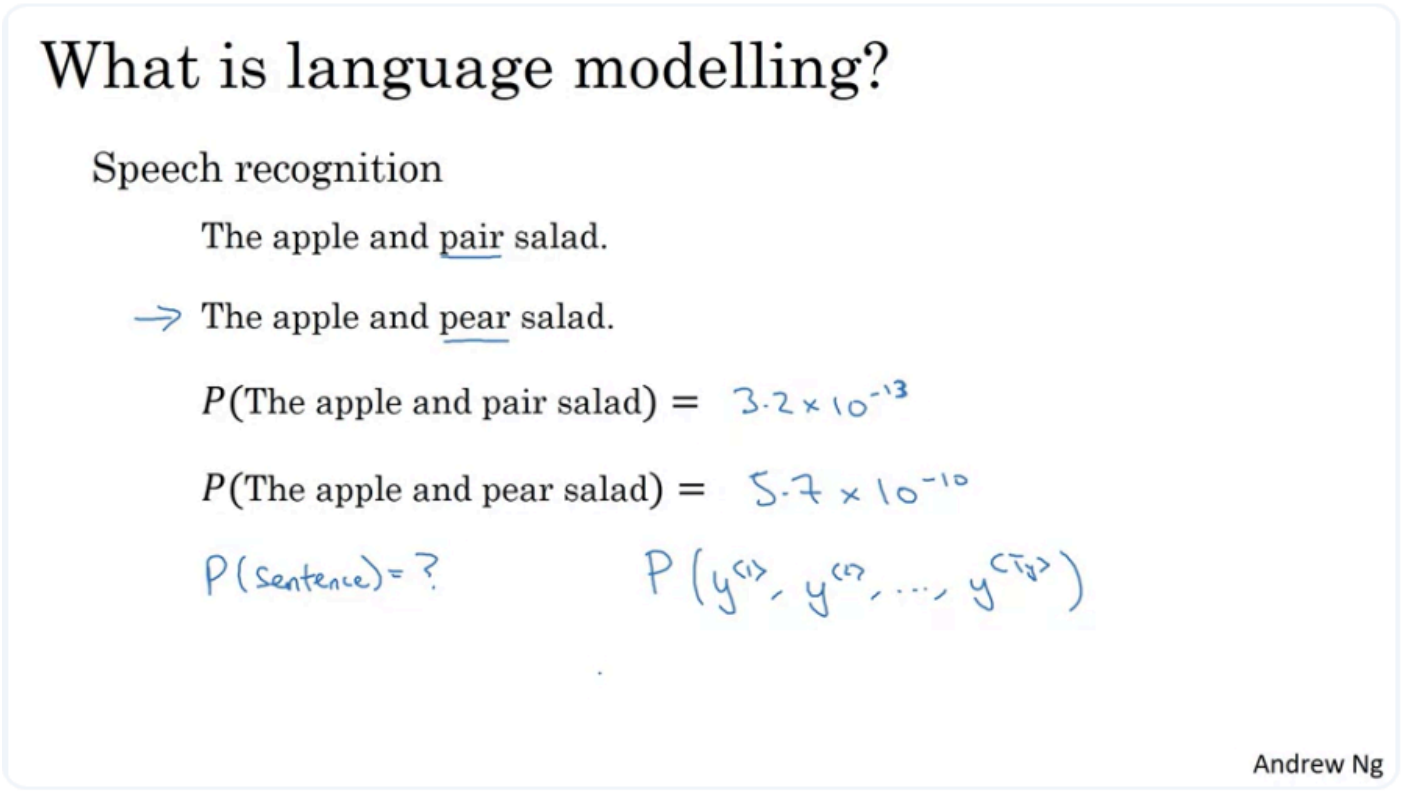
$y^1, y^2, ..., y^T$

the language model estimates:

$P(y^1, y^2, ..., y^T)$

Language models are useful because they help systems distinguish between sentences that sound similar but differ in meaning or naturalness.

Example: Speech Recognition



Consider these two sentences:

1. The apple and **pair** salad was delicious.
2. The apple and **pear** salad was delicious.

The words **pair** and **pear** sound identical, so the audio signal alone may not clearly distinguish them.

A language model might assign probabilities such as:

P(sentence 1) = 3.2×10^{-13}

P(sentence 2) = 5.7×10^{-10}

The second sentence is more than 10^3 times as likely as the first one.

Therefore, a speech recognition system would probably choose:

> The apple and pear salad was delicious.

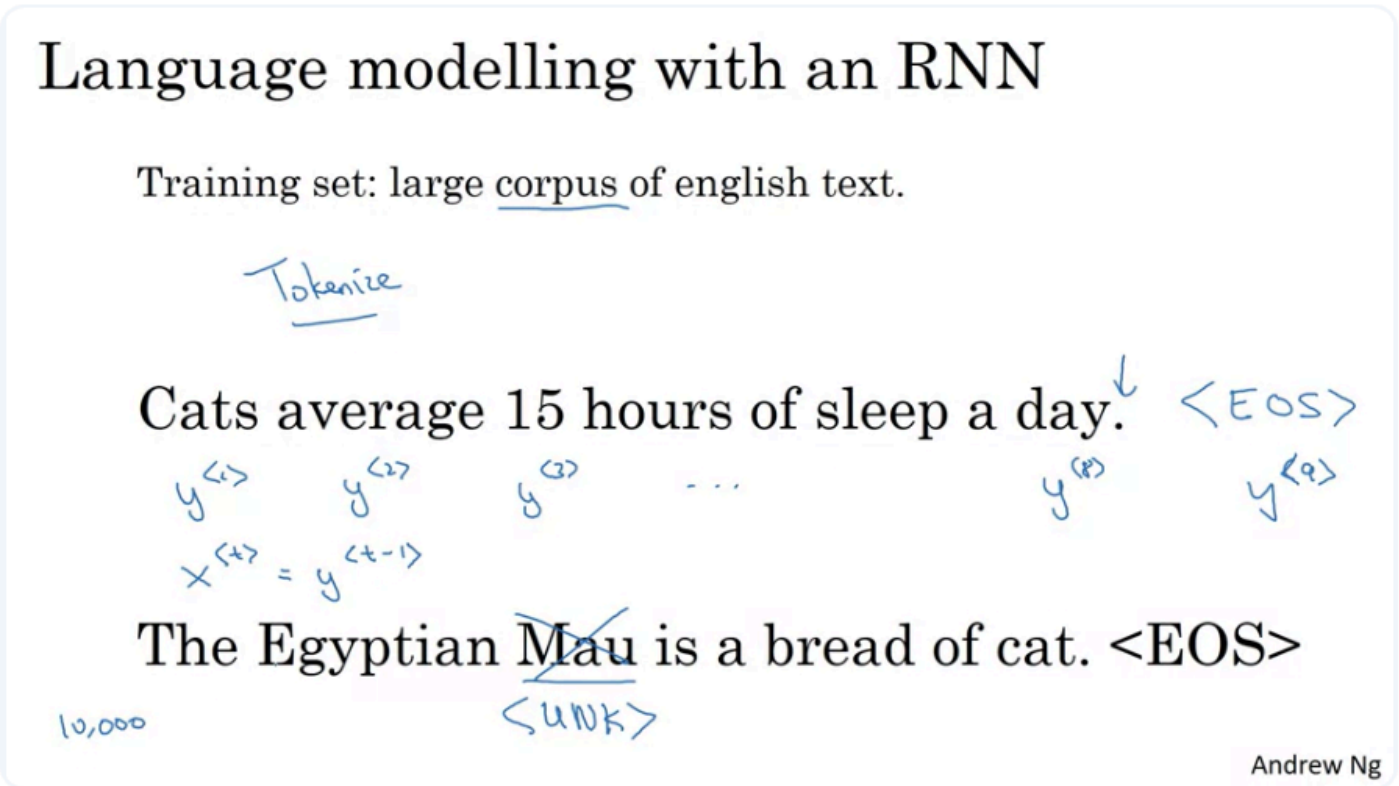
The language model provides linguistic context that complements the acoustic information.

Main Applications

Language models are important components of:

- Speech recognition systems
- Machine translation systems
- Text generation systems
- Autocomplete systems
- Spelling and grammar correction
- Conversational systems

In machine translation, the model helps ensure that the generated translation is not only accurate but also likely and natural in the target language.



Preparing Training Data

Text Corpus

To train a language model, we need a large collection of text in the target language.

This collection is called a **corpus**.

A corpus may contain:

- Books
- News articles
- Web pages
- Emails
- Conversations
- Documents
- Other collections of sentences

For example, suppose the training corpus contains the sentence:

> Cats average 15 hours of sleep a day.

Tokenization

The first preprocessing step is **tokenization**.

Tokenization divides a sentence into individual units called **tokens**.

Example:

Cats | average | 15 | hours | of | sleep | a | day

Each token is then mapped to:

- An integer index in the vocabulary, or
- A one-hot vector

A vocabulary may contain, for example, the 10,000 most common words in the training corpus.

Punctuation Handling

Punctuation can be processed in different ways.

Option 1: Ignore punctuation

Cats | average | 15 | hours | of | sleep | a | day

Option 2: Treat punctuation as separate tokens

Cats | average | 15 | hours | of | sleep | a | day | .

The choice depends on the task and the desired model behavior.

End-of-Sentence Token

A special token called **EOS** can be added to represent the end of a sentence.

EOS means:

> End of Sentence

Example:

Cats | average | 15 | hours | of | sleep | a | day | EOS

The EOS token helps the model learn when a sentence should stop.

Without EOS, the model learns to predict words but does not explicitly learn when the sequence ends.

Unknown Words

A fixed vocabulary cannot contain every possible word.

Suppose a model uses a vocabulary of the 10,000 most common words. A rare word, name, or technical term may not appear in that vocabulary.

Such words can be replaced with the special token:

UNK

UNK means:

> Unknown Word

For example:

> Mau is a breed of cat.

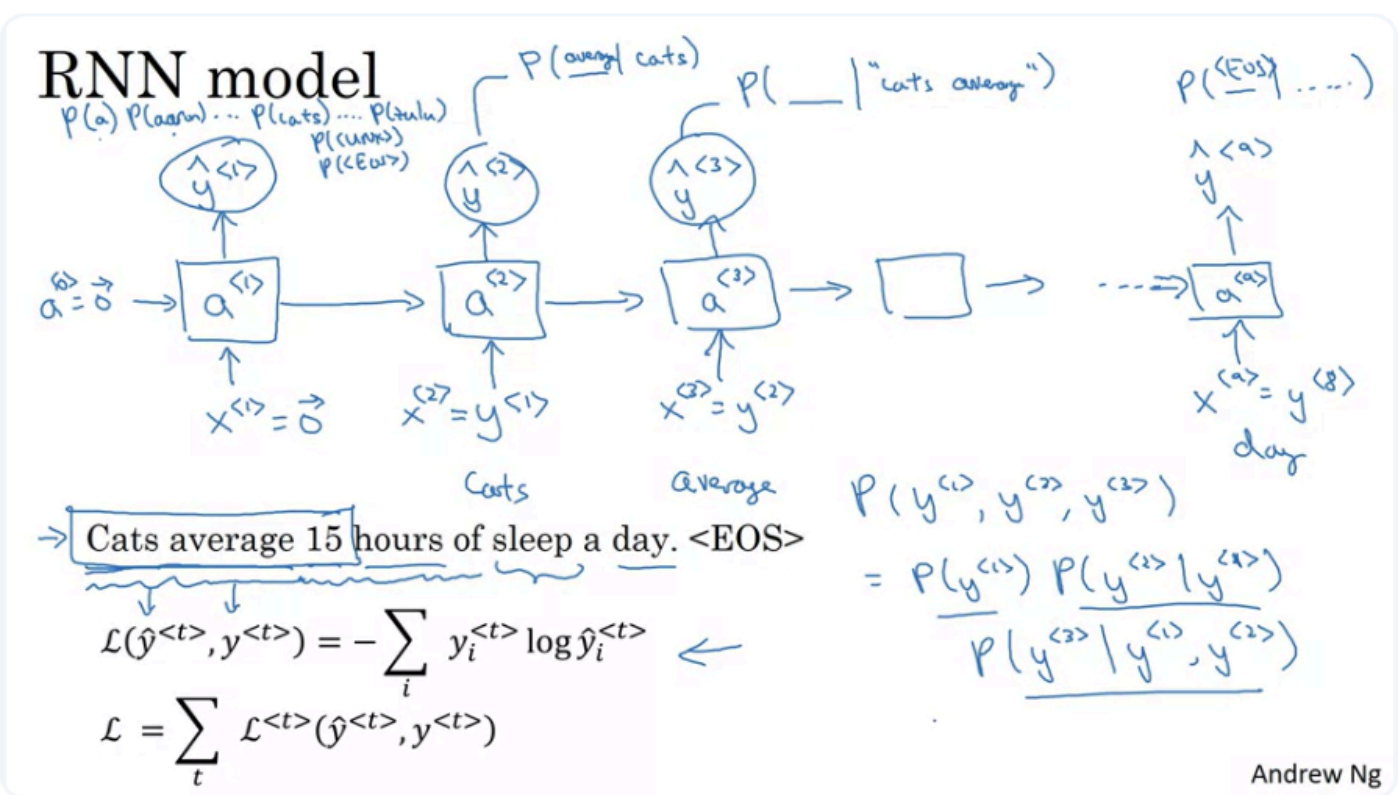
may be tokenized as:

UNK | is | a | breed | of | cat | EOS

The model then learns the probability of an unknown word rather than the probability of the specific rare word.

Modern language models often use subword tokenization, which reduces the need for an UNK token.

RNN Architecture for Language Modeling



Basic Objective

An RNN language model predicts one word at a time from left to right.

At each time step, it uses the previous words to predict the next word.

For example:

Cats → average → 15 → hours → of → sleep → a → day → EOS

The prediction at time step t can be written as:

$P(y^t \mid y^1, y^2, \ldots, y^{t-1})$

This means:

> The probability of the current word y^t given all previous words.

Initial State

At the first time step, there are no previous words.

Therefore, the initial hidden state is commonly initialized as a zero vector:

$a^0 = 0$

The first input may also be represented by a zero vector or a special beginning-of-sentence token:

$x^1 = 0$

The RNN calculates:

$a^1 = f(a^0, x^1)$

The hidden state a^1 is then used to predict the first word.

Predicting the First Word

The output layer applies a Softmax function:

$\hat{y}^1 = \text{Softmax}(W_y a^1 + b_y)$

The vector $\hat{y}^1$ contains a probability for every token in the vocabulary.

For a vocabulary with V words:

$\hat{y}^1 \in \mathbb{R}^V$

For example, the model estimates:

- P(first word = “a”)
- P(first word = “Aaron”)
- P(first word = “cats”)
- P(first word = “Zulu”)
- P(first word = UNK)
- P(first word = EOS)

For the running example, the correct first word is:

$y^1 = \text{cats}$

Predicting the Second Word

After the first word is known, it is provided as the input to the next time step:

$x^2 = y^1$

The RNN computes:

$a^2 = f(a^1, x^2)$

It then predicts:

$\hat{y}^2 = P(y^2 \mid y^1)$

Since the first word is **cats**, the model tries to predict the most likely word that follows it.

The correct second word is:

$y^2 = \text{average}$

Predicting Later Words

At the third step:

$x^3 = y^2$

The model predicts:

$\hat{y}^3 = P(y^3 \mid y^1, y^2)$

Given:

> Cats average

the correct next token is:

> 15

This process continues through the entire sentence.

At a general time step t:

$x^t = y^{t-1}$

The RNN predicts:

$\hat{y}^t = P(y^t \mid y^1, y^2, \ldots, y^{t-1})$

Predicting the End of the Sentence

After receiving the final regular word:

> day

the model should predict:

> EOS

Therefore:

P(EOS | cats, average, 15, hours, of, sleep, a, day)

should be high.

This allows the model to explicitly represent the probability that the sentence ends at that point.

Teacher Forcing

During training, the model receives the correct previous word as its next input.

For example:

x^2 = true y^1

x^3 = true y^2

x^4 = true y^3

This training method is commonly called **teacher forcing**.

Instead of using the model's own previous prediction, the correct word from the training data is supplied.

Teacher forcing usually makes training faster and more stable.

However, during text generation, the correct next word is unavailable. The model must use its own previously generated token as the next input.

Softmax Output

At every time step, the RNN produces a probability distribution over the vocabulary.

For vocabulary size V:

$\hat{y}^t = [p_1, p_2, \ldots, p_v]$

where:

$\Sigma_{i=1}^{V} p_i = 1$

Each value p_i represents the predicted probability of a vocabulary token being the next word.

For a vocabulary of 10,000 words, the model uses a 10,000-way Softmax output.

If special tokens are included, the output size may be slightly larger:

V = 10,000 regular words + UNK + EOS

Training Objective

Loss at One Time Step

At time step t:

- y^t is the true next word.
- $\hat{y}^t$ is the predicted probability distribution.

The model uses the Softmax cross-entropy loss:

$L^t = -\log P(y^t \mid y^1, \ldots, y^{t-1})$

This loss becomes small when the model assigns a high probability to the correct word.

It becomes large when the correct word receives a low probability.

Total Sequence Loss

The total loss for a sentence is the sum of the losses at every time step:

$L = \sum_{t=1}^{T} L^t$

Equivalently:

$L = -\sum_{t=1}^{T} \log P(y^t \mid y^1, \ldots, y^{t-1})$

The RNN is trained using backpropagation through time to minimize this loss over the full training corpus.

2. RNN cho language modeling

Trong language modeling, RNN phải dự đoán từ tiếp theo tại **mỗi time step**.

Ví dụ:

```
Input:   The  apple  is
Target:  apple  is   red
```

Tại mỗi bước t, model tạo phân phối xác suất:

$$\hat{y}^{\langle t \rangle} = P(y^{\langle t \rangle} \mid y^{\langle 1 \rangle}, \ldots, y^{\langle t-1 \rangle})$$

Loss tại time step t:

$$\mathcal{L}^{\langle t \rangle} = -\sum_{i=1}^{V} y_i^{\langle t \rangle} \log \hat{y}_i^{\langle t \rangle}$$

Trong đó V là kích thước vocabulary.

Vì vector nhãn $y^{\langle t \rangle}$ là one-hot, công thức có thể viết đơn giản thành:

$$\mathcal{L}^{\langle t \rangle} = -\log \hat{y}_{\text{target}}^{\langle t \rangle}$$

Total loss của toàn bộ chuỗi thường là tổng:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}^{\langle t \rangle}$$

Calculating Sentence Probability

A language model calculates the probability of a complete sentence using the chain rule of probability.

For a three-word sentence:

y^1, y^2, y^3

the probability is:

$P(y^1, y^2, y^3)$

$= P(y^1) \times P(y^2 \mid y^1) \times P(y^3 \mid y^1, y^2)$

For a general sequence:

$P(y^1, y^2, \ldots, y^T)$

$= \prod_{t=1}^{T} P(y^t \mid y^1, y^2, \ldots, y^{t-1})$

When EOS is included:

P(sentence)

$= P(y^1) \times P(y^2 \mid y^1) \times \ldots \times P(\text{EOS} \mid y^1, \ldots, y^T)$

Therefore, the probability of a sentence is obtained by multiplying the probabilities predicted at all time steps.

Why Log Probabilities Are Used

Sentence probabilities can become extremely small because many probabilities between 0 and 1 are multiplied together.

For numerical stability, implementations usually calculate the log probability:

$\log P(\text{sentence})$

$= \sum_{t=1}^{T} \log P(y^t \mid y^1, \ldots, y^{t-1})$

Multiplication in probability space becomes addition in log-probability space.

This helps avoid numerical underflow.

Model Workflow

Training Process

1. Collect a large text corpus.
2. Build a vocabulary.
3. Tokenize every sentence.
4. Add special tokens such as EOS and UNK when needed.
5. Initialize the RNN hidden state.
6. Provide the correct previous token as the next input.
7. Predict the next-token distribution using Softmax.
8. Calculate the cross-entropy loss at each time step.
9. Sum or average the losses across the sequence.
10. Update the parameters using backpropagation through time.

Inference Process

Given an initial sequence such as:

> Cats average 15 hours of

the model estimates:

P(next word | cats, average, 15, hours, of)

Possible predictions may include:

- sleep
- work
- rest
- activity
- exercise

The word **sleep** should receive a high probability because it naturally follows the context in the training example.

Key Mathematical Relationships

Input Relationship

During training:

$x^t = y^{t-1}$

Hidden-State Update

$a^t = f(W_{aa}a^{t-1} + W_{ax}x^t + b_a)$

Next-Word Prediction

$\hat{y}^t = \text{Softmax}(W_{ya}a^t + b_y)$

Conditional Probability

$\hat{y}^t$ represents:

$P(y^t \mid y^1, y^2, \ldots, y^{t-1})$

Sequence Probability

$P(y^1, \ldots, y^T)$

$= \prod_{t=1}^{T} P(y^t \mid y^1, \ldots, y^{t-1})$

Training Loss

$L = -\sum_{t=1}^{T} \log P(y^t \mid y^1, \ldots, y^{t-1})$

Key Takeaways

- A language model estimates how likely a sequence of words is.
- Language models support tasks such as speech recognition, translation, and text generation.
- An RNN language model predicts one token at a time from left to right.
- At time step t, the model predicts y^t based on all previous tokens.
- During training, the correct previous token is supplied as input using teacher forcing.
- A Softmax layer produces a probability distribution over the vocabulary.
- The total training loss is the sum of the next-token prediction losses.
- The probability of a complete sentence is calculated by multiplying its conditional token probabilities.
- EOS represents the end of a sentence.
- UNK represents words outside the model’s vocabulary.
- Once trained, the language model can assign probabilities to sentences or generate new sequences.

Further Learning

Sequence Sampling

Study how text is generated by repeatedly sampling from the model’s next-token distribution.

Important topics:

- Greedy decoding
- Random sampling
- Temperature
- Top-k sampling
- Top-p or nucleus sampling

Perplexity

Perplexity is a common metric for evaluating language models.

It measures how uncertain the model is when predicting the next token.

Lower perplexity generally indicates better next-token prediction.

For a sequence of length T:

Perplexity = exp(L ÷ T)

where L is the total negative log-likelihood.

Recurrent Architectures

Explore improved RNN variants that handle longer dependencies:

- Long Short-Term Memory networks
- Gated Recurrent Units
- Bidirectional RNNs

For left-to-right text generation, only previous context can be used, so the language model is normally unidirectional.

Modern Language Models

Modern language models usually replace RNN recurrence with Transformer-based self-attention.

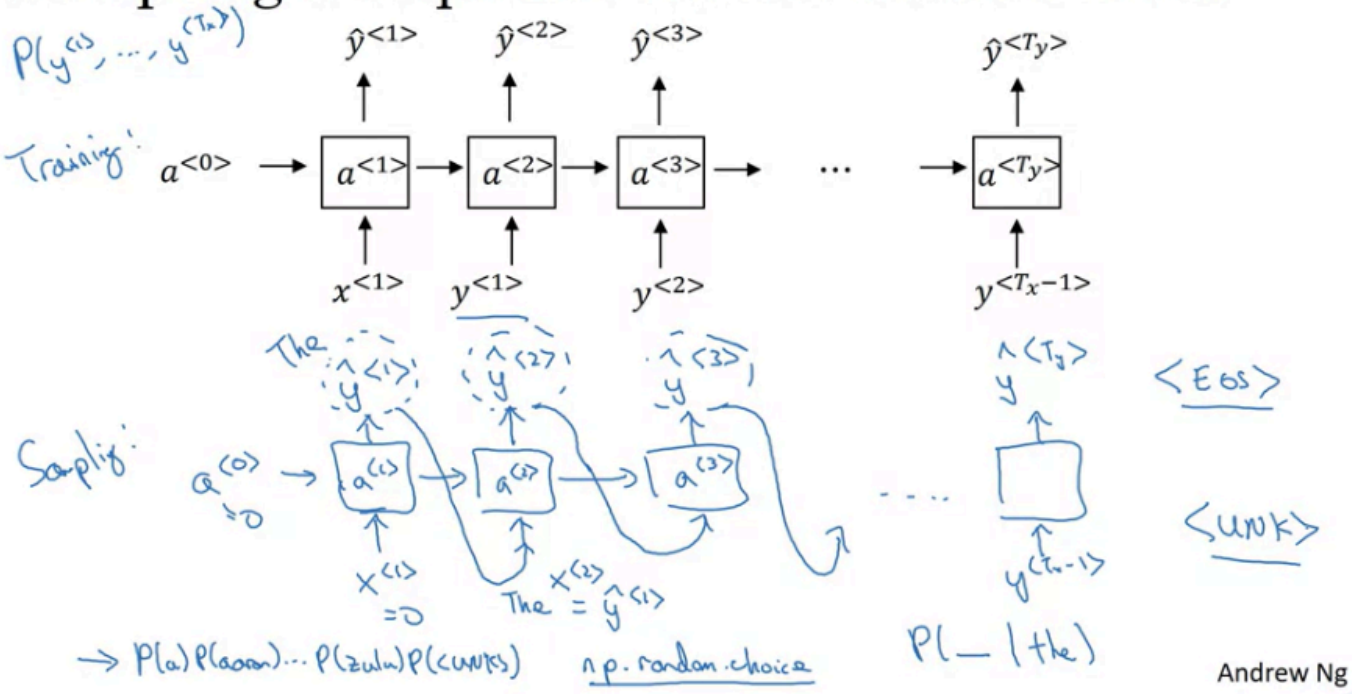
Useful topics:

- Causal self-attention
- Autoregressive language modeling
- Positional encoding
- Decoder-only Transformers
- GPT-style models

Recommended Resources

- *Sequence Models* from the Deep Learning Specialization
- *The Unreasonable Effectiveness of Recurrent Neural Networks* by Andrej Karpathy
- *Speech and Language Processing* by Daniel Jurafsky and James H. Martin
- PyTorch tutorials on character-level RNN language modeling
- Research on teacher forcing and exposure bias
- The original Transformer paper: *Attention Is All You Need*

Sampling Novel Sequences



Sampling Novel Sequences from an RNN Language Model

After training a sequence model, we can informally evaluate what it has learned by asking it to generate new sequences.

The model has learned a probability distribution over possible word sequences:

$P(y^1, y^2, \ldots, y^T)$

To generate a new sentence, we repeatedly sample one word from the model's predicted probability distribution and use that sampled word to predict the next one.

Training vs. Sequence Generation

During training, the model receives the correct previous word from the dataset:

$x^t = y^{t-1}$

During generation, the correct next word is unknown. Therefore, the model uses its own previously generated word as the next input:

$x^t = \hat{y}^{t-1}$

This means that generated words are fed back into the RNN one step at a time.

Step 1: Initialize the Model

At the first time step, there is no previous word or hidden state.

Initialize:

$x^1 = 0$

$a^0 = 0$

The RNN then produces a Softmax probability distribution for the first word:

$\hat{y}^1 = P(y^1)$

This distribution contains the probability of every possible token in the vocabulary, such as:

- "a"
- "Aaron"
- "the"
- "Zulu"
- UNK
- EOS

Step 2: Sample the First Word

Instead of always choosing the word with the highest probability, randomly sample a word according to the Softmax distribution.

For example, suppose the model predicts:

Word	Probability
the	0.30
a	0.20
cats	0.10
other words	0.40

The word **“the”** has the highest probability, but it is not guaranteed to be selected. Sampling preserves randomness and allows the model to generate different sequences.

In NumPy, this can be implemented using:

`np.random.choice()`

Conceptually:

sampled_word = np.random.choice(vocabulary, p=probabilities)

The argument `p` contains the probabilities produced by the Softmax output.

Step 3: Feed the Sampled Word Back into the RNN

Suppose the first sampled word is:

$\hat{y}^1$ = “the”

At the next time step, use this sampled word as the new input:

$x^2 = \hat{y}^1$

The RNN now estimates:

$\hat{y}^2 = P(y^2 \mid y^1 = \text{“the”})$

In other words, it predicts a probability distribution for the second word given that the first generated word was “the.”

A second word is then sampled from this new distribution.

Step 4: Repeat Autoregressively

The same process is repeated at every time step.

At time step 3:

$x^3 = \hat{y}^2$

The model predicts:

$\hat{y}^3 = P(y^3 \mid \hat{y}^1, \hat{y}^2)$

At a general time step t:

$x^t = \hat{y}^{t-1}$

and:

$\hat{y}^t \sim P(y^t \mid \hat{y}^1, \hat{y}^2, \ldots, \hat{y}^{t-1})$

The symbol $\sim$ means “sampled from.”

This generation process is called **autoregressive generation** because each new prediction depends on previously generated outputs.

Complete Sampling Procedure

1. Initialize $x^1 = 0$ and $a^0 = 0$.
2. Compute the Softmax distribution for the first token.
3. Randomly sample $\hat{y}^1$ from that distribution.
4. Set $x^2 = \hat{y}^1$.

5. Predict the probability distribution for the second token.
6. Sample $\hat{y}^2$.
7. Feed $\hat{y}^2$ into the next time step.
8. Continue until a stopping condition is reached.

The generated sequence may look like:

“the” → “cat” → “slept” → “on” → “the” → “chair” → EOS

When Should Generation Stop?

Option 1: Stop at the EOS Token

If the vocabulary contains the **EOS** token, continue sampling until EOS is generated.

Example:

the → cat → sleeps → all → day → EOS

EOS indicates that the model considers the sentence complete.

This produces sequences with variable lengths.

Option 2: Use a Fixed Maximum Length

If EOS is not included in the vocabulary, define a maximum number of time steps.

For example:

- Generate exactly 20 words
- Generate at most 100 words
- Stop after 50 characters

This prevents generation from continuing indefinitely.

A maximum length is often used even when EOS exists, in case the model fails to generate EOS.

Handling the UNK Token

The model may generate the special token:

UNK

UNK represents a word that is outside the vocabulary.

There are two common ways to handle it.

Keep UNK in the Output

The generated sequence may include UNK:

> The UNK walked across the street.

This is simple, but the sentence may be difficult to interpret.

Reject and Resample

When UNK is sampled:

1. Reject the sampled token.
2. Remove or mask the probability of UNK.
3. Renormalize the remaining probabilities.
4. Sample again.

This ensures that UNK never appears in the generated output.

Conceptually:

$P'(\text{UNK}) = 0$

Then normalize the remaining distribution so that:

$\Sigma P'(\text{word}) = 1$

Sampling vs. Greedy Decoding

Random Sampling

Random sampling chooses tokens according to their predicted probabilities.

For example:

- “the”: 50%
- “a”: 30%
- “one”: 20%

All three words may be selected, but more likely words are selected more frequently.

Advantages:

- Produces diverse outputs
- Generates different sentences across runs
- Better reflects the learned probability distribution

Disadvantages:

- May generate unlikely or incorrect words
- Errors can affect all later predictions

Greedy Decoding

Greedy decoding always selects the token with the highest probability:

$\hat{y}^t = \text{argmax } P(y^t \mid \text{previous tokens})$

It is more deterministic but often less diverse.

The transcript focuses on random sampling rather than greedy decoding.

Why Generated Sequences Can Differ

Because the algorithm samples randomly, the same trained model and initial state can produce different sentences.

For example:

Run 1:

> The cat sleeps near the window.

Run 2:

> A young cat walked into the house.

Run 3:

> The people were waiting outside.

The model does not retrieve a sentence directly from its training data. It constructs a new sequence by repeatedly sampling from conditional probability distributions.

Error Accumulation During Generation

During training, the model receives correct previous words.

During generation, it receives its own predictions.

If an unlikely or incorrect word is sampled, it becomes part of the context for every later prediction.

For example:

the → cat → drives → ...

After generating the unusual phrase “the cat drives,” the model must continue based on that context.

This difference between training and inference is related to **exposure bias**.

Key Equations

Initial Conditions

$x^1 = 0$

$a^0 = 0$

First Token

$\hat{y}^1 \sim P(y^1)$

Following Tokens

$x^t = \hat{y}^{t-1}$

$\hat{y}^t \sim P(y^t \mid \hat{y}^1, \ldots, \hat{y}^{t-1})$

Stopping Condition

Stop when:

$\hat{y}^t = \text{EOS}$

or:

t = maximum sequence length

Simplified Pseudocode

```
current_token = zero_vector
hidden_state = zero_vector
generated_tokens = []

for step in range(max_length):
    probabilities, hidden_state = rnn(
        current_token,
        hidden_state,
    )

    sampled_token = np.random.choice(
        vocabulary,
        p=probabilities,
    )

    if sampled_token == "UNK":
        continue

    if sampled_token == "EOS":
        break

    generated_tokens.append(sampled_token)
    current_token = sampled_token
```

In a real implementation, the sampled token is usually converted to an index, one-hot vector, or embedding before being passed into the next time step.

Key Takeaways

- A trained RNN language model can generate new sequences by sampling one token at a time.
- The first input and hidden state are usually initialized with zeros.

- At each time step, the Softmax layer produces a probability distribution over the vocabulary.
- A token is randomly sampled from this distribution.
- The sampled token becomes the input to the next time step.
- This process is autoregressive because every generated token depends on previous generated tokens.
- Generation can stop when EOS is sampled or when a fixed maximum length is reached.
- UNK can either remain in the output or be rejected and resampled.
- Random sampling produces more diverse outputs than always choosing the most likely token.
- Generated errors can accumulate because the model conditions future predictions on its own outputs.

Further Learning

Temperature Sampling

Temperature controls how random the sampling distribution is.

For logits z_i:

$P_i = \text{Softmax}(z_i \div \tau)$

where τ is the temperature.

- $\tau < 1$: More confident and conservative output
- $\tau = 1$: Original model distribution
- $\tau > 1$: More random and diverse output

Top-k Sampling

Only the k most probable tokens are kept before sampling.

This prevents the model from selecting extremely unlikely words.

Top-p Sampling

Top-p, or nucleus sampling, keeps the smallest group of tokens whose total probability is at least p.

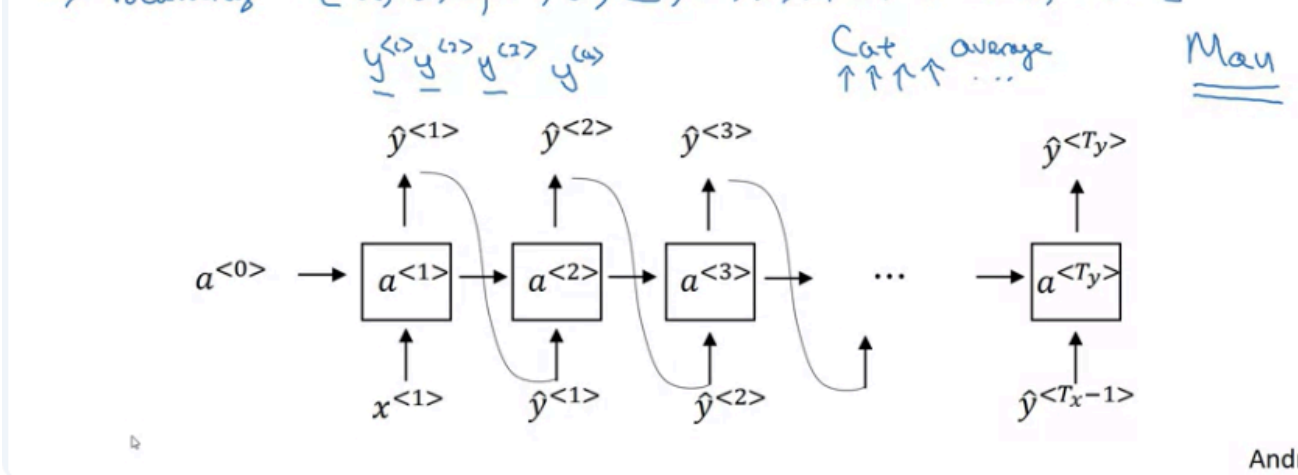
This dynamically adjusts the number of candidate words at each time step.

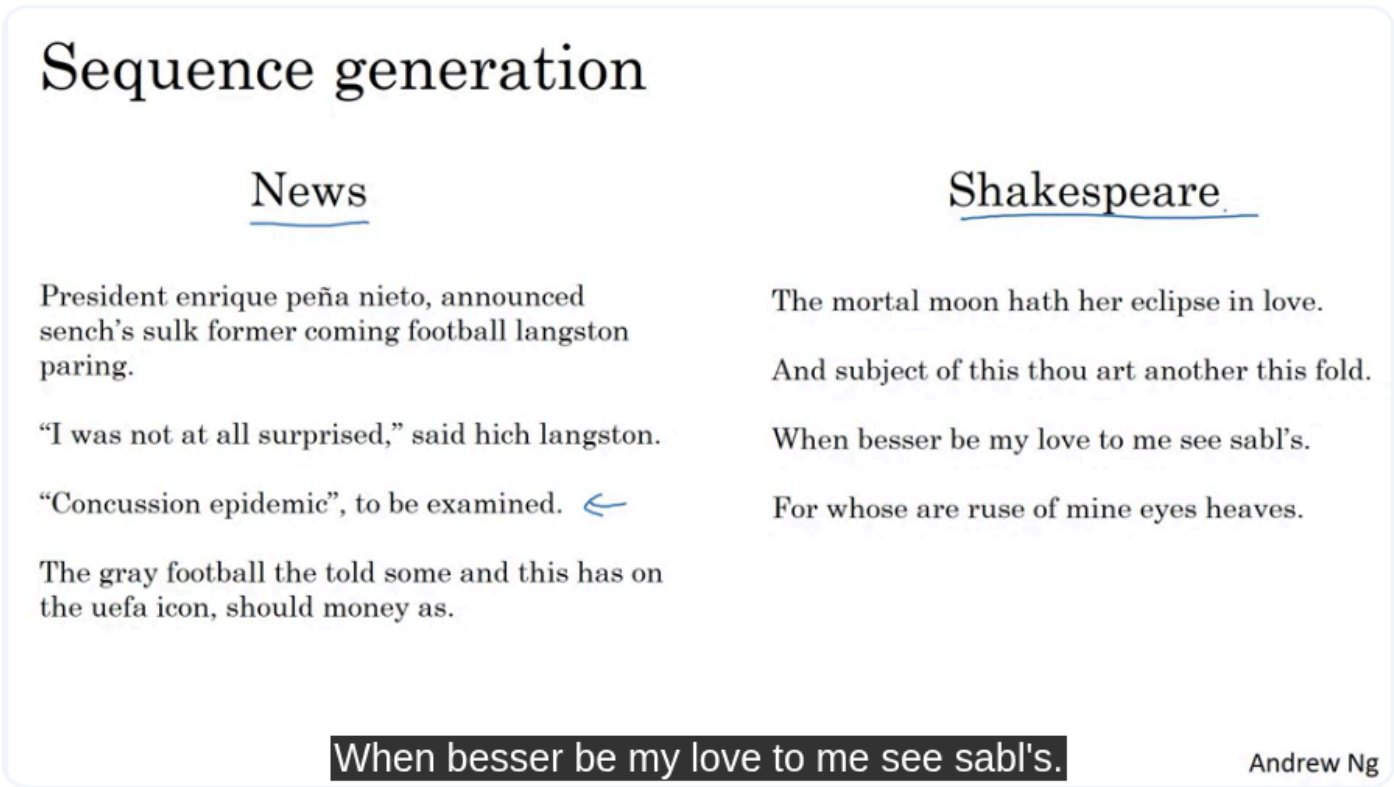
Exposure Bias

Study why using correct tokens during training but generated tokens during inference can cause errors to accumulate.

Related topics include:

- Teacher forcing
- Scheduled sampling
- Free-running generation





Word-Level vs. Character-Level RNN Language Models

So far, the language model has used **words** as its basic units. However, an RNN language model can also operate at the **character level**.

The main difference is the definition of the vocabulary and the length of the input sequences.

Word-Level Language Model

In a word-level model, each token represents a complete word.

Example sentence:

> Cats average 15 hours of sleep a day.

The sequence is:

y^1 = “cats”

y^2 = “average”

y^3 = “15”

y^4 = “hours”

...

y^8 = “day”

The vocabulary may contain thousands or tens of thousands of words.

For example:

V = 10,000 words

Special tokens such as **UNK** and **EOS** may also be included.

Character-Level Language Model

In a character-level model, each token represents a single character rather than a complete word.

The vocabulary may contain:

- Lowercase letters: a–z
- Uppercase letters: A–Z
- Digits: 0–9
- Spaces
- Punctuation
- Line breaks
- Other characters appearing in the training data

A simple way to build the vocabulary is to collect every unique character that appears in the training corpus.

Example

For the sentence:

> cats average 15 hours of sleep a day

the sequence begins as:

y^1 = “c”

y^2 = “a”

y^3 = “t”

y^4 = “s”

y^5 = “ ”

y^6 = “a”

y^7 = “v”

...

The space is treated as a token because it helps the model learn where words begin and end.

The model predicts one character at a time:

$P(y^t \mid y^1, y^2, ..., y^{t-1})$

For example, after receiving:

> c a

the model may assign a high probability to:

> t

Advantages of Character-Level Models

No Unknown Words

A character-level model usually does not need an **UNK** token.

Even when the model encounters a word that never appeared in the training data, it can still represent that word as a sequence of known characters.

For example, suppose the rare word:

> mau

is not included in a word-level vocabulary.

A word-level model may replace it with:

> UNK

A character-level model can represent it directly:

m → a → u

Therefore, it can assign a non-zero probability to new words, names, spelling variations, and rare vocabulary.

Small Vocabulary

Character-level vocabularies are usually much smaller than word-level vocabularies.

A word-level model may use:

V ≈ 10,000–100,000 tokens

A character-level model may use only:

V ≈ 30–200 characters

This makes the Softmax output layer smaller.

Flexible Word Generation

Because the model generates characters directly, it can produce:

- New words
- Rare names
- Technical terms
- Spelling variations
- Morphologically complex words

This can be useful when the application contains many uncommon or changing terms.

Disadvantages of Character-Level Models

Much Longer Sequences

A sentence with only 10–20 words may contain dozens or hundreds of characters.

For example:

Word-level sequence:

> cats | average | 15 | hours | of | sleep | a | day

Character-level sequence:

> c | a | t | s | $_{SP}$ | a | v | e | r | a | g | e | …

Therefore:

Tcharacter ≫ Tword

where T represents sequence length.

The RNN must process many more time steps for the same sentence.

Difficulty Capturing Long-Range Dependencies

Because sequences are much longer, information must travel through many recurrent steps.

For example, a dependency between two distant words may correspond to dozens of character-level time steps.

This makes it harder for a basic RNN to learn relationships between:

- The beginning and end of a sentence
- Subjects and verbs
- Pronouns and earlier nouns
- Long phrases or paragraphs

This problem is closely related to the **vanishing gradient problem**.

Higher Computational Cost

Character-level models require more recurrent operations because each sentence contains more tokens.

As a result, they are generally:

- Slower to train
- Slower to generate from
- More computationally expensive
- Harder to optimize for long sequences

Even though the character vocabulary is smaller, the greater number of time steps often makes training more expensive overall.

Comparison

Aspect	Word-Level Model	Character-Level Model
Basic token	Complete word	Individual character
Vocabulary size	Large	Small
Sequence length	Shorter	Much longer
Unknown words	Requires UNK or subwords	Usually unnecessary
New word generation	Limited	Flexible
Long-range dependencies	Easier to capture	Harder to capture
Computational cost	Usually lower	Usually higher
Typical use	General language modeling	Specialized applications

When to Use Character-Level Models

Character-level models may be useful when:

- Unknown words appear frequently
- The vocabulary changes often
- The data contains many names
- Spelling is important
- The language has complex word formation
- The model must generate new words
- The data uses a specialized vocabulary
- The application involves source code or structured text
- The dataset contains many misspellings or informal expressions

Possible applications include:

- Name generation
- Spelling correction
- Handwriting recognition

- Source-code generation
- Morphological analysis
- Domain-specific text generation
- Modeling languages without clear word boundaries

When to Use Word-Level Models

Word-level models are generally more suitable when:

- The vocabulary is relatively stable
- Efficient training is important
- Long-range semantic relationships matter
- The dataset contains standard natural language
- Unknown words are not a major problem

Historically, word-level models were more widely used because character-level models required much more computation.

Modern Alternative: Subword Tokenization

Modern language models commonly use **subword tokens**, which provide a compromise between word-level and character-level modeling.

A word may be divided into smaller reusable units.

For example:

> unbelievable

could be tokenized as:

un | believe | able

Subword tokenization provides several benefits:

- Fewer unknown words
- Shorter sequences than character-level models
- Smaller vocabularies than word-level models
- Better handling of rare and newly formed words

Common subword methods include:

- Byte-Pair Encoding
- WordPiece
- Unigram Language Model
- SentencePiece
- Byte-level tokenization

Subword tokenization is now more common than pure word-level or pure character-level tokenization in large language models.

Sampling from a Character-Level Model

The sampling process is the same as for a word-level RNN, but each output is a character.

Initial Step

Initialize:

$x^{\langle 1 \rangle} = 0$

$a^{\langle 0 \rangle} = 0$

The model predicts a probability distribution over all characters:

$\hat{y}^1 = P(y^1)$

A character is sampled from this distribution.

Following Steps

The sampled character becomes the next input:

$x^t = \hat{y}^{t-1}$

The model then predicts:

$\hat{y}^t \sim P(y^t \mid \hat{y}^1, \ldots, \hat{y}^{t-1})$

This continues until:

- An EOS token is generated, or
- A maximum number of characters is reached

The sampled characters are concatenated to form words and sentences.

Generated Text Examples

A character-level language model can imitate the style of its training corpus.

News-Article Model

When trained on news articles, the model may generate text such as:

> Concussion epidemic to be examined.

The result may be imperfect or ungrammatical, but it can still resemble the style and vocabulary of news writing.

Shakespeare Model

When trained on Shakespearean text, the model may generate lines such as:

> The mortal moon hath her eclipse in love.

The output may not always be fully meaningful, but it can reproduce patterns such as:

- Archaic vocabulary
- Sentence rhythm
- Punctuation style
- Character names
- Poetic phrasing

This demonstrates that the model learns statistical patterns from the training corpus.

What the Model Learns

A character-level RNN does not memorize grammar rules explicitly.

Instead, it learns patterns such as:

- Which characters commonly follow others
- How words are formed
- Where spaces and punctuation appear
- Which phrases are common
- What writing style is present in the corpus

With enough training, it can generate text that resembles the original data, even if the output is not always grammatically or semantically correct.

Limitations of Basic RNNs

Basic RNNs struggle to remember information over long sequences.

During backpropagation through many time steps, gradients may become extremely small.

This is called the:

Vanishing gradient problem

As a result, a basic RNN may learn short-term patterns effectively but fail to capture long-term relationships.

For character-level models, this issue is especially important because a single sentence may contain many time steps.

More Powerful Recurrent Architectures

The limitations of basic RNNs motivate the use of improved architectures.

GRU

GRU stands for:

Gated Recurrent Unit

A GRU uses gates to control:

- Which information should be remembered
- Which information should be updated
- Which information should be forgotten

GRUs are designed to preserve useful information over longer sequences.

LSTM

LSTM stands for:

Long Short-Term Memory

An LSTM introduces a memory cell and several gates that help information and gradients flow across many time steps.

The main gates are:

- Forget gate
- Input gate
- Output gate

LSTMs are especially useful for learning long-range dependencies.

Key Takeaways

- RNN language models can operate at either the word level or character level.
- In a word-level model, each y^t represents a complete word.
- In a character-level model, each y^t represents one character.
- Character-level models avoid the unknown-word problem.
- They can generate rare or completely new words.
- Their vocabularies are smaller than word-level vocabularies.
- However, character-level sequences are much longer.
- Longer sequences make training slower and increase difficulty in learning long-range dependencies.
- Character-level models can imitate the style of a training corpus, such as news or Shakespearean writing.
- Modern systems often use subword tokenization as a compromise.
- The vanishing gradient problem limits basic RNNs.
- GRUs and LSTMs were developed to handle longer-term dependencies more effectively.

Further Learning

Vanishing and Exploding Gradients

Study how gradients behave during backpropagation through time.

Important topics:

- Repeated multiplication of gradients
- Gradient clipping
- Weight initialization
- Long-term dependencies

Gated Recurrent Unit

Learn how the update and reset gates help a GRU preserve information.

Useful concepts:

- Update gate
- Reset gate
- Candidate hidden state
- Hidden-state interpolation

Long Short-Term Memory

Study how an LSTM uses a memory cell and gates to manage information flow.

Useful concepts:

- Cell state
- Forget gate
- Input gate
- Output gate
- Candidate memory

Tokenization Methods

Compare different tokenization levels:

- Character-level
- Word-level
- Subword-level
- Byte-level

A useful practical exercise is to compare the same sentence under each tokenization method and measure the resulting vocabulary size and sequence length.

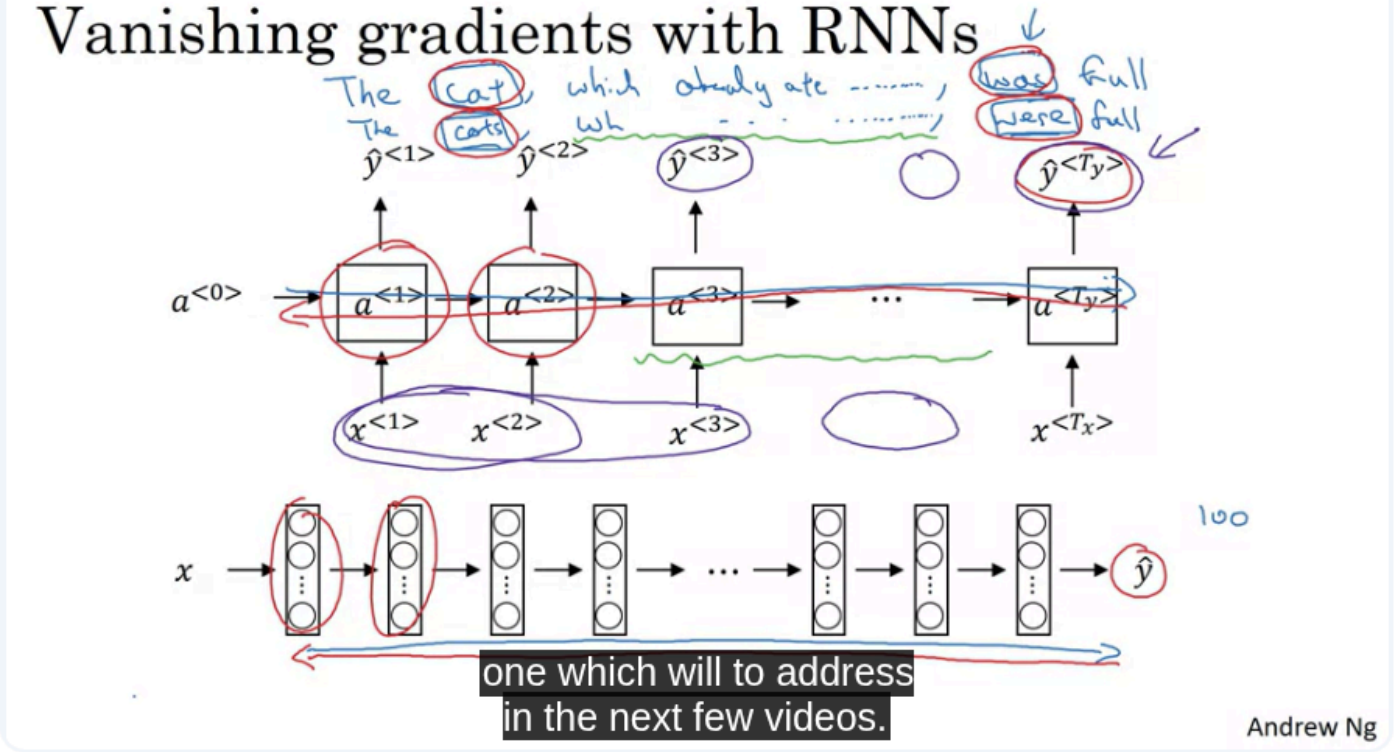
Character-Level RNN Implementation

A useful project is to train a character-level RNN on:

- Shakespeare
- Song lyrics
- News headlines
- Source code
- Vietnamese poetry

Then compare the generated output at different sampling temperatures.

Vanishing Gradients with RNNs



Vanishing Gradients in Recurrent Neural Networks

Basic RNNs can be trained using **Backpropagation Through Time**, but they often struggle to learn relationships between events that are far apart in a sequence.

The main cause is the **vanishing gradient problem**.

Long-Term Dependencies

A long-term dependency occurs when information from an early part of a sequence is needed much later.

Consider these examples:

> The **cat**, which already ate a large amount of delicious food, **was** full.

> The **cats**, which already ate a large amount of delicious food, **were** full.

To choose between **was** and **were**, the model must remember whether the earlier subject was singular or plural.

The important relationship is:

cat → was

cats → were

However, many words may appear between the subject and the verb:

> The cat, which already ate a large amount of food that was prepared earlier in the day, was full.

The information about **cat** must be preserved across many time steps before it is used.

This is an example of a long-range dependency.

RNNs as Very Deep Neural Networks

An RNN can be unfolded across time.

For a sequence with T time steps, the unfolded RNN behaves similarly to a neural network with T layers.

For example:

- 100 time steps ≈ a 100-layer neural network
- 1,000 time steps ≈ a 1,000-layer neural network
- 10,000 time steps ≈ a 10,000-layer neural network

During forward propagation, information moves from left to right:

$x^1 \rightarrow a^1 \rightarrow a^2 \rightarrow \ldots \rightarrow a^T$

During backpropagation, gradients move from right to left:

$L^{T} \rightarrow a^{T} \rightarrow a^{T-1} \rightarrow \ldots \rightarrow a^{1}$

When the sequence is long, the gradients must pass through many recurrent computations.

The Vanishing Gradient Problem

During backpropagation, gradients are repeatedly multiplied by weight matrices and activation derivatives.

A simplified expression is:

$\partial L \div \partial a^{t} = \partial L \div \partial a^{T} \times \prod_{k=t+1}^{T} \partial a^{k} \div \partial a^{k-1}$

The symbol $\prod$ means repeated multiplication.

If most of these derivatives have magnitudes smaller than 1, repeated multiplication causes the gradient to decrease exponentially.

For example:

$0.5^{10} = 0.0009765625$

$0.5^{100} \approx 7.9 \times 10^{-31}$

The gradient can become almost zero.

When this happens, parameters associated with earlier time steps receive almost no useful learning signal.

Practical Consequences

Suppose the model incorrectly predicts:

> The cats ... **was** full.

The error occurs near the end of the sequence.

To correct this mistake, the gradient must propagate backward to the point where the word **cats** was processed.

If the gradient vanishes before reaching that earlier step, the model cannot effectively learn that:

> cats should later be followed by were

As a result, a basic RNN mainly learns **local dependencies**.

Its predictions are influenced more strongly by recent inputs than by distant inputs.

Conceptually:

$P(y^{t} \mid x^{t-1}, x^{t-2})$

is easier to learn than:

$P(y^{t} \mid x^{1}, x^{2}, \ldots, x^{t-1})$

when the relevant information appears very early.

Local Influence in Basic RNNs

The hidden state is theoretically designed to summarize the full previous sequence:

$a^{t} = f(a^{t-1}, x^{t})$

However, in practice, information from distant time steps may gradually disappear.

Therefore:

- A nearby word can strongly influence the current output.
- A distant word may have only a weak influence.
- Early information may be overwritten by newer information.
- The model may fail to preserve grammatical or semantic context.

This limits the ability of basic RNNs to model:

- Subject–verb agreement

- Long sentences
- Long conversations
- Document-level context
- Long audio sequences
- Long-term temporal patterns

Why Vanishing Gradients Occur

Consider a simplified RNN hidden-state equation:

$a^t = \tanh(W_{aa}a^{t-1} + W_{ax}x^t + b_a)$

During backpropagation, the gradient includes repeated terms involving:

W_{aa}

and:

$\tanh'(z)$

The derivative of tanh satisfies:

$0 \leq \tanh'(z) \leq 1$

If the product of the recurrent weights and activation derivatives repeatedly has magnitude below 1, the gradient shrinks at every time step.

A simplified relationship is:

gradient at early step ≈ gradient at later step × c^n

where:

- $0 < c < 1$
- n is the number of recurrent steps

As n increases:

$c^n \to 0$

Therefore, the gradient vanishes exponentially.

Exploding Gradients

Gradients do not always decrease. They may also grow exponentially.

This is called the **exploding gradient problem**.

If repeated multiplication involves values larger than 1:

$c^n \to \infty$ when $c > 1$

For example:

$2^{10} = 1{,}024$

$2^{100} \approx 1.27 \times 10^{30}$

Extremely large gradients can cause parameter updates to become unstable.

The update rule is:

$\theta \leftarrow \theta - \alpha \nabla_\theta L$

If $\nabla_\theta L$ is extremely large, the new parameter values may become excessively large or invalid.

Signs of Exploding Gradients

Exploding gradients are often easier to detect than vanishing gradients.

Common symptoms include:

- Loss suddenly becoming extremely large
- Unstable training

- Model weights becoming very large
- Predictions changing dramatically
- Numerical overflow
- NaN values

NaN means:

> Not a Number

NaNs may appear when numerical operations exceed the range that the computer can represent.

Gradient Clipping

A common solution for exploding gradients is **gradient clipping**.

Gradient clipping limits the size of the gradient before updating the parameters.

Clipping by Value

Each gradient component is restricted to a fixed range:

$g_i = \min(\max(g_i, -c), c)$

For example, with $c = 5$:

- 12 becomes 5
- −9 becomes −5
- 3 remains 3

Clipping by Norm

A more common method is to limit the total gradient norm.

If:

$\| g \| > c$

then rescale the gradient:

$g \leftarrow c \times g \div \| g \|$

where:

- g is the gradient vector
- $\| g \|$ is its norm
- c is the maximum allowed norm

This preserves the gradient direction while reducing its magnitude.

Example

Suppose:

$\| g \| = 20$

and the maximum norm is:

$c = 5$

Then:

$g \leftarrow 5 \times g \div 20$

Therefore, every gradient component is multiplied by:

$5 \div 20 = 0.25$

The new gradient norm becomes 5.

PyTorch Example

```python
import torch

loss.backward()

torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=5.0,
)

optimizer.step()
optimizer.zero_grad()
```

Gradient clipping is normally applied after `loss.backward()` and before `optimizer.step()`.

Exploding vs. Vanishing Gradients

Aspect	Vanishing gradients	Exploding gradients
Gradient behavior	Approaches zero	Becomes extremely large
Main effect	Early layers or steps learn slowly	Training becomes unstable
Detection	Often difficult	Usually easy
Common symptoms	Poor long-term memory	NaNs and rapidly increasing loss
Basic solution	Gated architectures	Gradient clipping
Severity	Gradual and subtle	Potentially catastrophic

Why Gradient Clipping Does Not Solve Vanishing Gradients

Gradient clipping limits gradients that are too large.

It does not increase gradients that are already close to zero.

For example:

g = 0.0000001

Applying a maximum norm of 5 does nothing because the gradient is already below the threshold.

Therefore:

- Gradient clipping is effective for exploding gradients.
- It does not address the fundamental cause of vanishing gradients.

Solving vanishing gradients requires changing the architecture so that information and gradients can flow more directly through time.

Solutions to Vanishing Gradients

Gated Recurrent Unit

A **Gated Recurrent Unit**, or **GRU**, uses gates to control the hidden state.

The gates help the network decide:

- What information to keep
- What information to update
- What information to forget

GRUs create pathways through which important information can be preserved across many time steps.

Long Short-Term Memory

An **LSTM** introduces a separate cell state that can carry information over long distances.

It uses gates such as:

- Forget gate
- Input gate
- Output gate

These gates regulate how information enters, remains in, and leaves the memory cell.

Better Initialization

Careful recurrent weight initialization can reduce gradient instability.

Examples include:

- Orthogonal initialization
- Identity initialization
- Proper variance scaling

However, initialization alone does not fully solve long-term dependency problems.

Alternative Activation Functions

Activation functions can influence gradient flow.

However, simply replacing tanh does not usually solve all recurrent vanishing-gradient problems because repeated recurrent multiplication remains.

Residual and Skip Connections

Skip connections can create shorter paths for gradients.

These techniques are more commonly associated with deep feedforward networks and modern sequence architectures.

Attention Mechanisms

Attention allows a model to directly access earlier positions rather than compressing the entire past into one recurrent hidden state.

This provides a shorter path between distant tokens.

Transformers extend this idea by replacing recurrence with self-attention.

Intuition Behind Gated Memory

Suppose the model sees:

> The cats ...

It needs to preserve the information:

subject number = plural

A gated architecture can maintain this information across the intervening words:

plural → plural → plural → ... → plural

When the model reaches the verb, it can use the stored information to predict:

> were

The important idea is to create an almost unchanged memory path through time.

If the information can pass through with only small modifications, the gradient can also flow backward through that path more effectively.

Key Equations

Basic RNN Update

$a^{t} = \tanh(W_{aa}a^{t-1} + W_{ax}x^{t} + b_{a})$

Parameter Update

$$\theta \leftarrow \theta - \alpha \nabla_\theta L$$

Gradient Through Time

$$\partial L \div \partial a^t = \partial L \div \partial a^T \times \prod_{k=t+1}^{T} \partial a^k \div \partial a^{k-1}$$

Vanishing Condition

If:

$$\left|\partial a^k \div \partial a^{k-1}\right| < 1$$

across many steps, then:

gradient → 0

Exploding Condition

If:

$$\left|\partial a^k \div \partial a^{k-1}\right| > 1$$

across many steps, then:

gradient → ∞

Gradient Clipping by Norm

$$g \leftarrow c \times g \div \|g\|, \text{ if } \|g\| > c$$

Key Takeaways

- Basic RNNs struggle with long-term dependencies.
- An RNN unfolded over many time steps behaves like a very deep neural network.
- Gradients must propagate backward through every recurrent time step.
- Repeated multiplication can make gradients decrease exponentially.
- Vanishing gradients prevent early time steps from receiving useful learning signals.
- Basic RNNs therefore tend to focus on recent inputs and local dependencies.
- Gradients can also increase exponentially, producing exploding gradients.
- Exploding gradients may cause unstable loss, very large parameters, or NaNs.
- Gradient clipping is a practical and effective solution for exploding gradients.
- Gradient clipping does not solve vanishing gradients.
- GRUs and LSTMs use gated memory mechanisms to preserve information and gradient flow over longer sequences.

Further Learning

Backpropagation Through Time

Study how a recurrent model is unfolded and differentiated across time.

Important topics:

- Shared parameters across time steps
- Truncated Backpropagation Through Time
- Computational graphs
- Memory requirements

Gradient Norm Monitoring

Track gradient norms during training to diagnose instability.

Useful measurements include:

- Total gradient norm

- Maximum gradient value
- Gradient distribution by layer
- Number of NaN or infinite values

GRU Architecture

The next important topic is how GRUs use:

- Update gates
- Relevance or reset gates
- Candidate hidden states

The update gate creates a direct mechanism for preserving information across many time steps.

LSTM Architecture

Study how the LSTM cell state supports nearly additive information flow, which helps gradients survive over longer sequences.

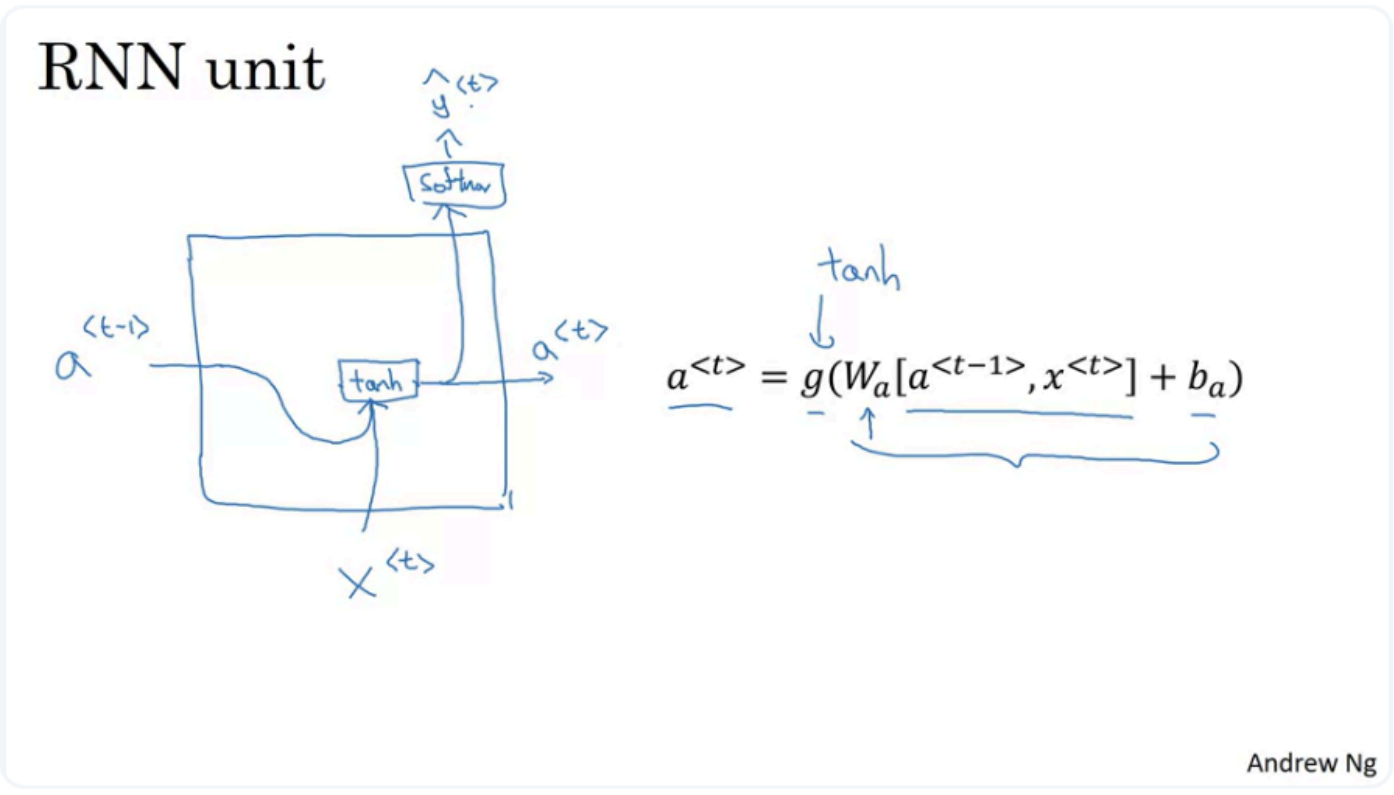
Modern Alternatives

Compare recurrent architectures with:

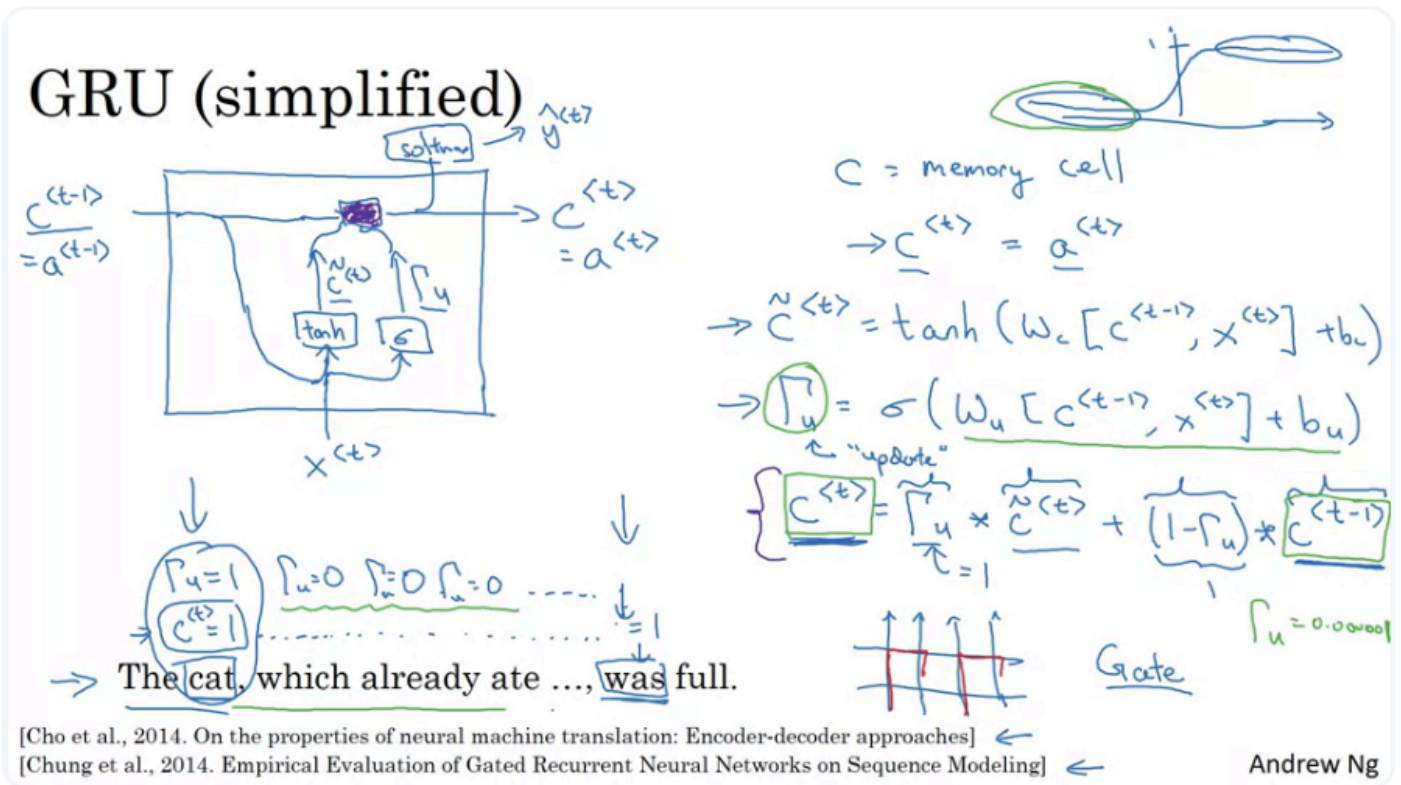
- Attention
- Self-attention
- Transformers
- State-space models

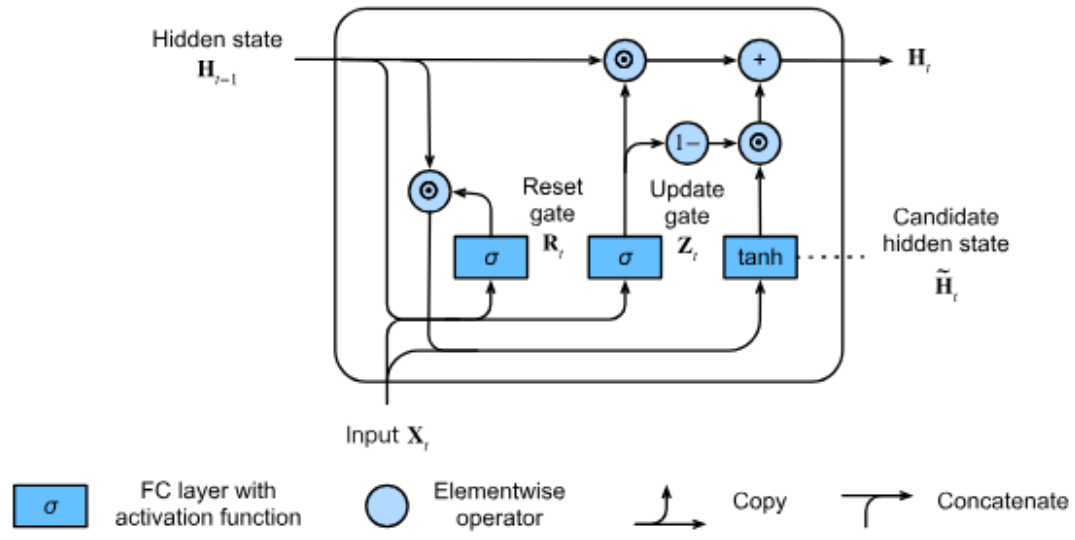
These architectures provide alternative approaches to modeling long-range dependencies.

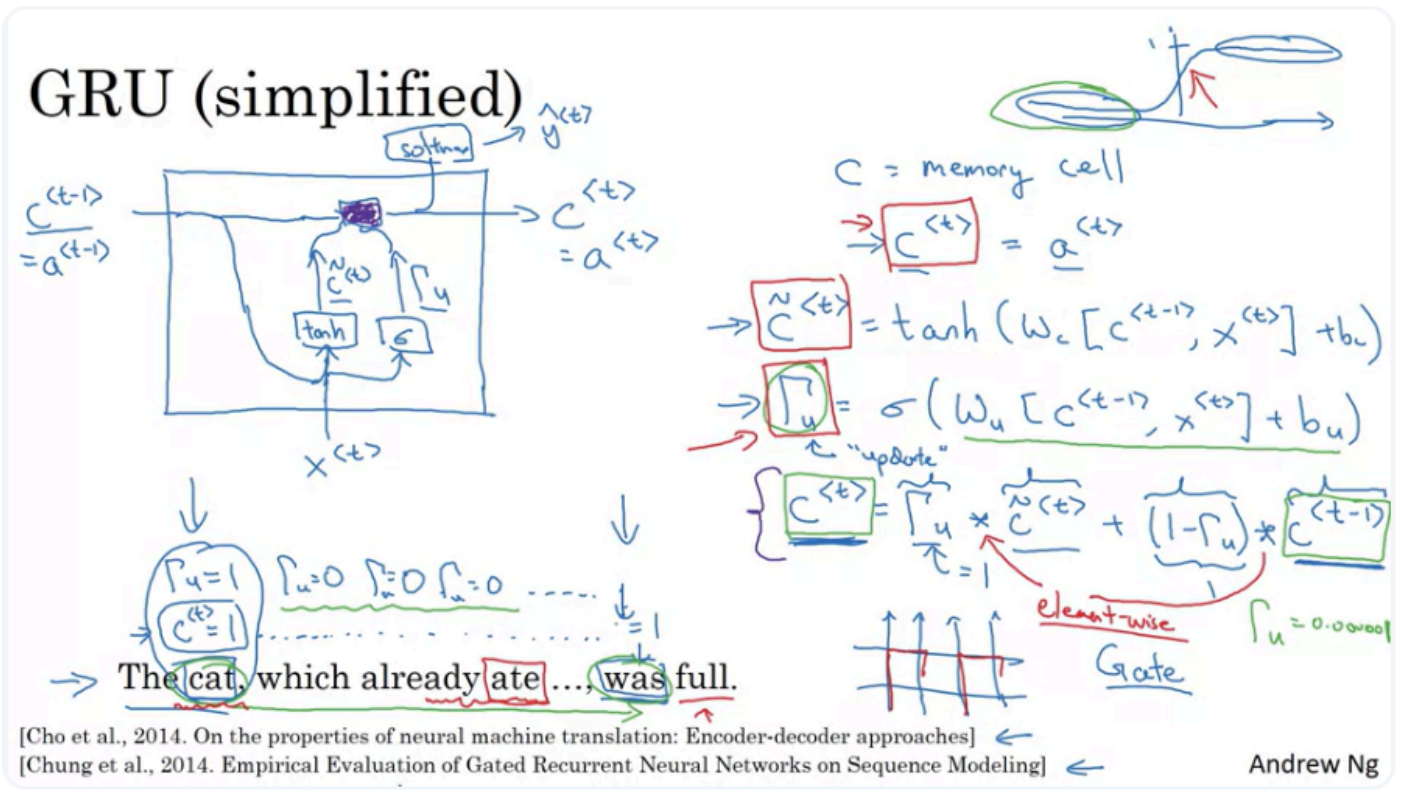
Gated Recurrent Unit (GRU)



Normal RNN unit is above


GRU (simplified)
softmax
tanh
The cat, which already ate ..., was full.
[Cho et al., 2014. On the properties of neural machine translation: Encoder-decoder approaches]
[Chung et al., 2014. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling]
Andrew Ng


Hidden state
Reset gate
Update gate
Candidate hidden state
tanh
Input
FC layer with activation function
Elementwise operator
Copy
Concatenate


GRU (simplified)
softmax
tanh
The cat, which already ate ..., was full.
element-wise
Gate
[Cho et al., 2014. On the properties of neural machine translation: Encoder-decoder approaches]
[Chung et al., 2014. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling]
Andrew Ng

Full GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

LSTM

The cat, which ate already, was full.

Andrew Ng

Gated Recurrent Unit

A **Gated Recurrent Unit**, or **GRU**, is a modified RNN architecture designed to:

- Capture long-range dependencies
- Preserve important information across many time steps
- Reduce the vanishing gradient problem

The main idea is to add a memory mechanism and gates that control when information should be updated or preserved.

From a Basic RNN to a GRU

In a basic RNN, the hidden state is computed as:

$a^t = \tanh(W_a[a^{t-1}, x^t] + b_a)$

where:

- a^{t-1} is the previous hidden state
- x^t is the current input
- W_a is the parameter matrix
- b_a is the bias
- a^t is the new hidden state

The hidden state may then be passed to a Softmax layer:

$\hat{y}^t = \text{Softmax}(W_Y a^t + b_Y)$

A basic RNN can model local patterns well, but it often loses information from much earlier time steps.

Motivation: Long-Range Dependencies

Consider the sentence:

> The cat, which already ate a large amount of delicious food, was full.

The model needs to remember that:

> cat = singular

This information is required much later to correctly choose:

> was

For a plural subject:

> The cats, which already ate a large amount of food, were full.

the model must remember:

> cats = plural

and later choose:

> were

A GRU can preserve such information in its memory across many intervening words.

Memory Cell

A GRU introduces a memory variable:

C^t

C^t represents the information stored at time step t.

In the GRU notation used in the lecture:

$a^t = C^t$

Therefore, the GRU hidden activation and memory cell have the same value.

This differs from an LSTM, where the hidden state and cell state are separate.

The memory cell can store information such as:

- Whether the subject is singular or plural
- The topic currently being discussed
- Important semantic information
- Long-term context from earlier in the sequence

Candidate Memory Value

At each time step, the GRU computes a possible new value for the memory cell.

This is called the **candidate memory value**:

$\tilde{C}^t$

For the simplified GRU:

$\tilde{C}^t = \tanh(W_c[C^{t-1}, x^t] + b_c)$

where:

- C^{t-1} is the previous memory
- x^t is the current input
- W_c and b_c are learned parameters
- $\tilde{C}^t$ is a candidate for replacing the previous memory

The symbol ~ indicates that this is only a candidate value, not necessarily the final memory value.

Update Gate

The GRU uses an **update gate (Gamma of u)**:

Γ_u

The update gate determines whether the memory should be replaced with the new candidate value or preserved from the previous time step.

It is computed using a sigmoid function:

$\Gamma_u = \sigma(W_u[C^{t-1}, x^t] + b_u)$

Because sigmoid outputs values between 0 and 1:

$0 \leq \Gamma_u \leq 1$

For intuition, Γ_u can be thought of as being close to either:

- 1: update the memory
- 0: keep the old memory

In practice, it may also take values between 0 and 1.

Memory Update Equation

The new memory cell is computed as:

$C^t = \Gamma_u \odot \tilde{C}^t + (1 - \Gamma_u) \odot C^{t-1}$

where $\odot$ means element-wise multiplication.

This equation combines:

- The new candidate memory $\tilde{C}^t$
- The previous memory C^{t-1}
- The update gate Γ_u

When the Update Gate Is 1

If:

$\Gamma_u = 1$

then:

$C^t = \tilde{C}^t$

The old memory is replaced by the new candidate value.

This may happen when the model reads an important word such as:

> cat

At this point, the model may store:

> subject = singular

When the Update Gate Is 0

If:

$\Gamma_u = 0$

then:

$C^t = C^{t-1}$

The memory remains unchanged.

This allows the model to preserve information while processing many unimportant intermediate words.

For example:

cat → which → already → ate → a → lot → of → food → was

The singular information can remain stored until it is needed to predict **was**.

Intermediate Gate Values

If:

$0 < \Gamma_u < 1$

the GRU combines the old and new memory.

For example:

$\Gamma_u = 0.2$

gives:

$C^t = 0.2\tilde{C}^t + 0.8C^{t-1}$

This means the model mostly preserves the old memory while making a small update.

How the GRU Reduces Vanishing Gradients

Suppose the update gate remains very close to zero:

$\Gamma_u \approx 0$

Then:

$C^t \approx C^{t-1}$

The memory can pass almost unchanged through many time steps:

$C^t \approx C^{t-1} \approx C^{t-2} \approx \ldots \approx C^1$

This creates a nearly direct path for information and gradients through the sequence.

As a result:

- Important information is less likely to disappear
- Gradients can propagate over longer distances
- The model can learn relationships between distant words

The gate can become extremely close to zero when the sigmoid input is strongly negative.

For example:

$\Gamma_u = 0.000001$

Then the previous memory is preserved almost exactly.

Vector-Valued Memory

The memory cell is not usually a single number.

If the hidden dimension is 100:

$C^t \in \mathbb{R}^{100}$

Then:

- $\tilde{C}^t \in \mathbb{R}^{100}$
- $\Gamma_u \in \mathbb{R}^{100}$

The update gate contains a separate value for every memory dimension.

Conceptually:

$\Gamma_u = [0, 1, 0, 0.8, \ldots]$

Each dimension decides independently whether to:

- Keep the old value
- Replace it
- Partially update it

For example:

- One dimension may store singular or plural information
- Another may store whether the topic involves food
- Another may store sentiment
- Another may track sentence structure

This allows the GRU to update only selected parts of its memory at each time step.

Element-Wise Multiplication

The memory update uses element-wise multiplication:

$C^t = \Gamma_u \odot \tilde{C}^t + (1 - \Gamma_u) \odot C^{t-1}$

For example:

$\Gamma_u = [1, 0, 0.5]$

$\tilde{C}^t = [4, 8, 6]$

$C^{t-1} = [2, 3, 10]$

Then:

$C^t = [1\times4 + 0\times2, 0\times8 + 1\times3, 0.5\times6 + 0.5\times10]$

$C^t = [4, 3, 8]$

Therefore:

- The first component is fully updated
- The second component is preserved
- The third component is partially updated

Full GRU

The simplified GRU uses only the update gate.

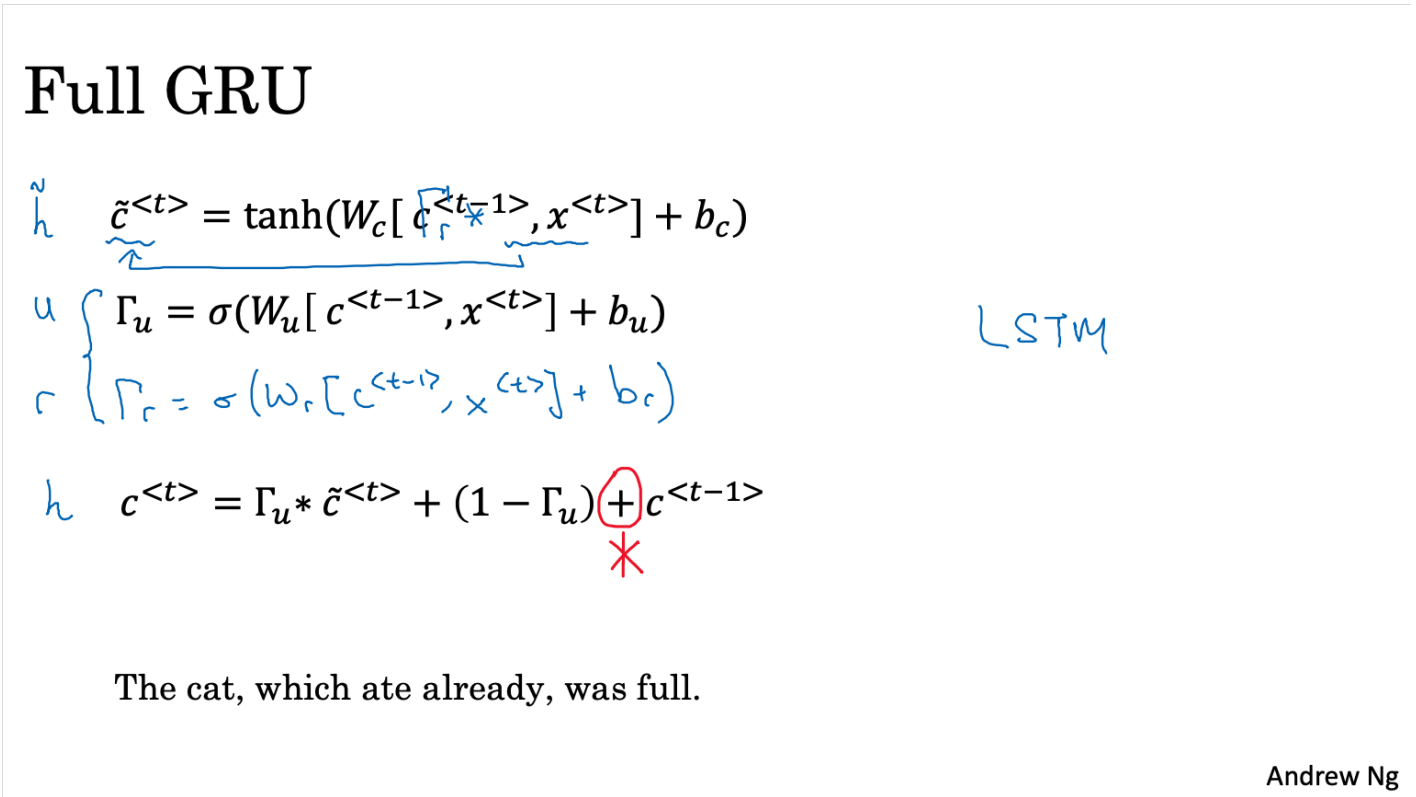
The full GRU introduces another gate:

Γ_r

This is commonly called the **reset gate**. In the lecture, it is described as a **relevance gate**.

The reset gate determines how relevant the previous memory is when computing the candidate memory.

Reset or Relevance Gate



The reset gate is computed as:

$\Gamma_r = \sigma(W_r[C^{t-1}, x^t] + b_r)$

It controls how much of the previous memory should be used when calculating $\tilde{C}^t$.

The full candidate equation becomes:

$\tilde{C}^t = \tanh(W_c[\Gamma_r \odot C^{t-1}, x^t] + b_c)$

When Γ_r Is Close to 0

If:

$\Gamma_r \approx 0$

then the old memory is mostly ignored when computing the candidate.

The model behaves as if it is starting a new context.

This may be useful when:

- A new sentence begins

- The topic changes
- Previous information is no longer relevant

When Γ_r Is Close to 1

If:

$\Gamma_r \approx 1$

then the previous memory strongly influences the candidate value.

This allows the new candidate to incorporate earlier context.

Complete GRU Equations

The standard GRU can be summarized using three equations.

Update Gate

$\Gamma_u = \sigma(W_u[C^{t-1}, x^t] + b_u)$

Reset Gate

$\Gamma_r = \sigma(W_r[C^{t-1}, x^t] + b_r)$

Candidate Memory

$\tilde{C}^t = \tanh(W_c[\Gamma_r \odot C^{t-1}, x^t] + b_c)$

Final Memory

$C^t = \Gamma_u \odot \tilde{C}^t + (1 - \Gamma_u) \odot C^{t-1}$

Hidden Activation

$a^t = C^t$

Depending on the textbook or software library, the update equation may appear with the roles of Γ_u and $1 - \Gamma_u$ reversed. The underlying idea remains the same: interpolate between the old state and the candidate state.

Intuition for the Two Gates

Update Gate Γ_u

Answers:

> Should the memory be updated?

It controls the balance between:

- New information
- Existing memory

Reset Gate Γ_r

Answers:

> How much previous memory is relevant when creating the new candidate?

It controls whether the candidate memory should depend strongly on the old state.

Example: Remembering Subject Number

Consider:

> The cats, which ate all the food in the kitchen, were full.

A simplified GRU behavior could be:

At “cats”

The update gate opens:

$\Gamma_u \approx 1$

The model stores:

subject number = plural

During the Middle Clause

For words such as:

> which ate all the food in the kitchen

the update gate stays nearly closed:

$\Gamma_u \approx 0$

The stored plural information remains unchanged.

At “were”

The model uses the preserved memory to predict the correct plural verb.

After the information has been used, the model may update or reset the relevant memory dimensions.

Simplified GRU vs. Full GRU

Component	Simplified GRU	Full GRU
Memory cell	Yes	Yes
Update gate	Yes	Yes
Reset gate	No	Yes
Candidate uses old state	Directly	Controlled by reset gate
Long-term memory	Supported	Supported
Typical practical use	Mainly educational	Standard implementation

Alternative Notation

GRU notation varies across textbooks and papers.

The following symbols may refer to similar quantities:

Lecture notation	Common notation
C^t	h^t
$\tilde{C}^t$	$\tilde{h}^t$
Γ_u	z^t
Γ_r	r^t

A common alternative formulation is:

z^t = update gate

r^t = reset gate

$\tilde{h}^t$ = candidate hidden state

h^t = final hidden state

The exact symbols are less important than understanding the function of each gate.

Why GRUs Work Well

GRUs are effective because they provide:

- A direct path for carrying information forward
- A mechanism for deciding when to update memory
- A mechanism for ignoring irrelevant past information

- Better gradient flow across long sequences
- More selective memory updates than basic RNNs

This makes them more capable of learning long-range dependencies.

GRU Output

Since:

$a^t = C^t$

the memory state can be used to produce an output:

$\hat{y}^t = \text{Softmax}(W_Y C^t + b_Y)$

Depending on the application, the output may represent:

- The next word in language modeling
- A token label in named entity recognition
- A class prediction
- An acoustic or temporal feature
- A sequence output

GRU Parameters

A full GRU learns separate parameters for:

- Candidate memory
- Update gate
- Reset gate
- Output layer

These include:

W_c, b_c

W_u, b_u

W_r, b_r

W_Y, b_Y

All parameters are learned through Backpropagation Through Time.

Computational Flow

At each time step:

1. Read the current input x^t.
2. Read the previous memory C^{t-1}.
3. Compute the update gate Γ_u.
4. Compute the reset gate Γ_r.
5. Use Γ_r to calculate the candidate memory $\tilde{C}^t$.
6. Combine $\tilde{C}^t$ and C^{t-1} using Γ_u.
7. Set $a^t = C^t$.
8. Optionally compute an output $\hat{y}^t$.

GRU Pseudocode

```
update_gate = sigmoid(
    W_update @ concat(previous_state, current_input)
    + b_update
```

```
)

reset_gate = sigmoid(
    W_reset @ concat(previous_state, current_input)
    + b_reset
)

candidate_state = tanh(
    W_candidate @ concat(
        reset_gate * previous_state,
        current_input,
    )
    + b_candidate
)

current_state = (
    update_gate * candidate_state
    + (1 - update_gate) * previous_state
)
```

The multiplication operations between gates and states are element-wise.

Key Differences: Basic RNN and GRU

Aspect	Basic RNN	GRU
Hidden state	Single recurrent state	Gated memory state
Update control	No explicit control	Update gate
Past-state relevance	Always included	Reset gate controls it
Long-term memory	Weak	Stronger
Vanishing gradients	Common	Reduced
Number of parameters	Lower	Higher
Training cost	Lower	Moderately higher
Long-range dependencies	Difficult	More effective

GRU and LSTM

GRUs and LSTMs are two widely used gated recurrent architectures.

GRU

- Uses update and reset gates
- Uses one main state
- Has fewer parameters
- Is relatively simple and computationally efficient

LSTM

- Uses a separate cell state and hidden state
- Uses input, forget, and output gates
- Has more parameters
- Provides more explicit control over memory

Neither is always better in every task. Their performance depends on:

- Dataset size
- Sequence length

- Available computation
- Task complexity
- Hyperparameter choices

Key Takeaways

- A GRU is designed to improve the memory capability of a basic RNN.
- It introduces a memory cell C^t.
- In a GRU, the hidden state and memory cell are the same: $a^t = C^t$.
- The candidate memory $\tilde{C}^t$ represents possible new information.
- The update gate Γ_u decides whether to preserve or replace the memory.
- When Γ_u is close to zero, the old memory can pass through almost unchanged.
- This direct memory path helps reduce vanishing gradients.
- The full GRU adds a reset or relevance gate Γ_r.
- The reset gate controls how much previous information is used to create the candidate state.
- Gates are vectors, so different memory dimensions can be updated independently.
- GRUs are effective at learning long-range dependencies.
- GRUs and LSTMs are the most common gated alternatives to basic RNNs.

Further Learning

LSTM Architecture

The next topic is the Long Short-Term Memory network.

Important concepts include:

- Cell state
- Hidden state
- Forget gate
- Input gate
- Output gate
- Candidate cell value

GRU Research Papers

The GRU architecture is closely associated with research by:

- Kyunghyun Cho
- Junyoung Chung
- Caglar Gulcehre
- Yoshua Bengio

Useful papers include:

- *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*
- *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*

Practical Experiments

A useful exercise is to compare:

- Basic RNN
- GRU

- LSTM

on the same sequential task.

Possible measurements include:

- Training loss
- Validation loss
- Accuracy
- Gradient norm
- Training speed
- Ability to remember distant information

PyTorch GRU

A GRU layer can be created using:

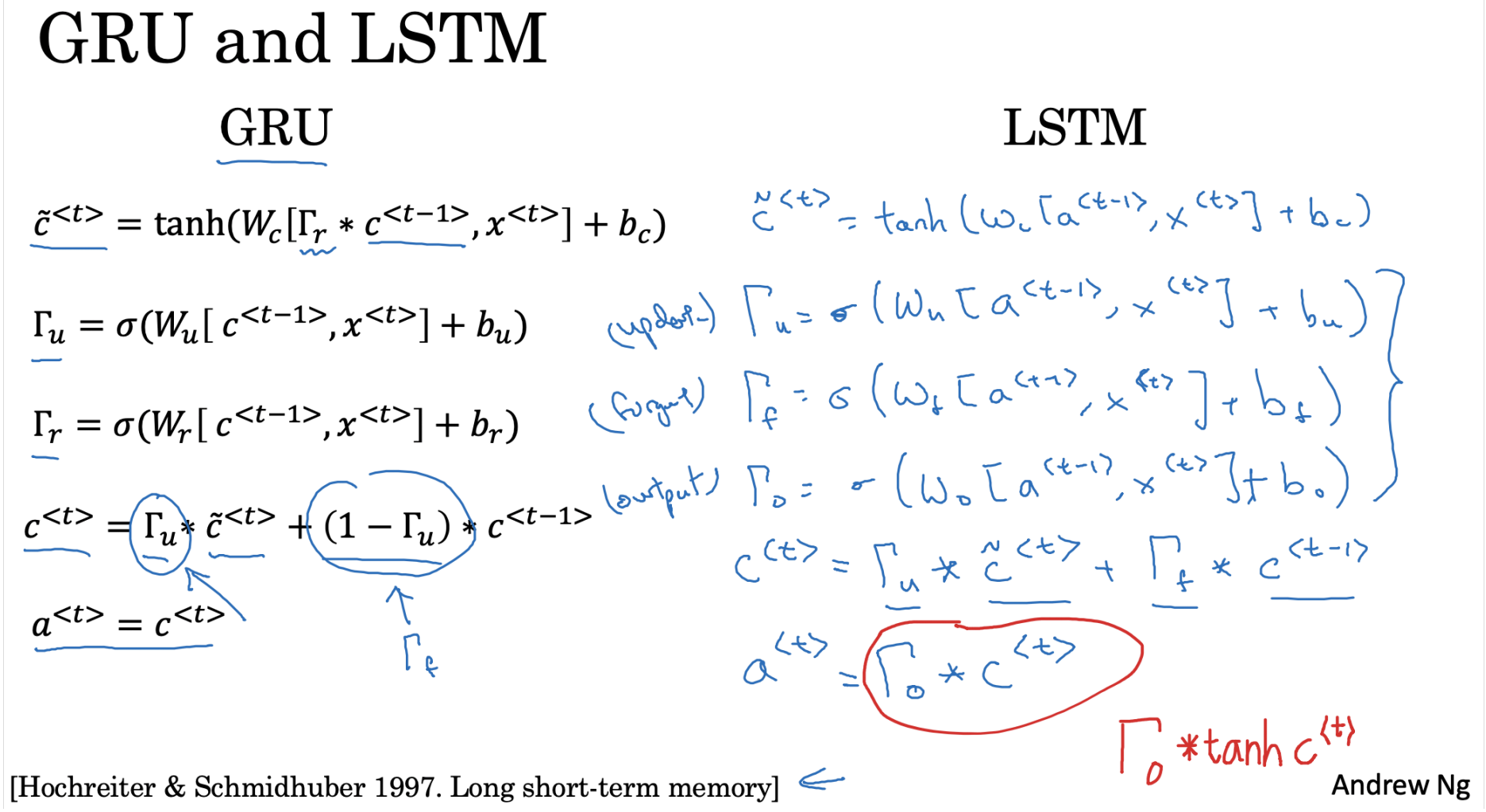
```
import torch.nn as nn

gru = nn.GRU(
    input_size=input_dim,
    hidden_size=hidden_dim,
    num_layers=1,
    batch_first=True,
)
```

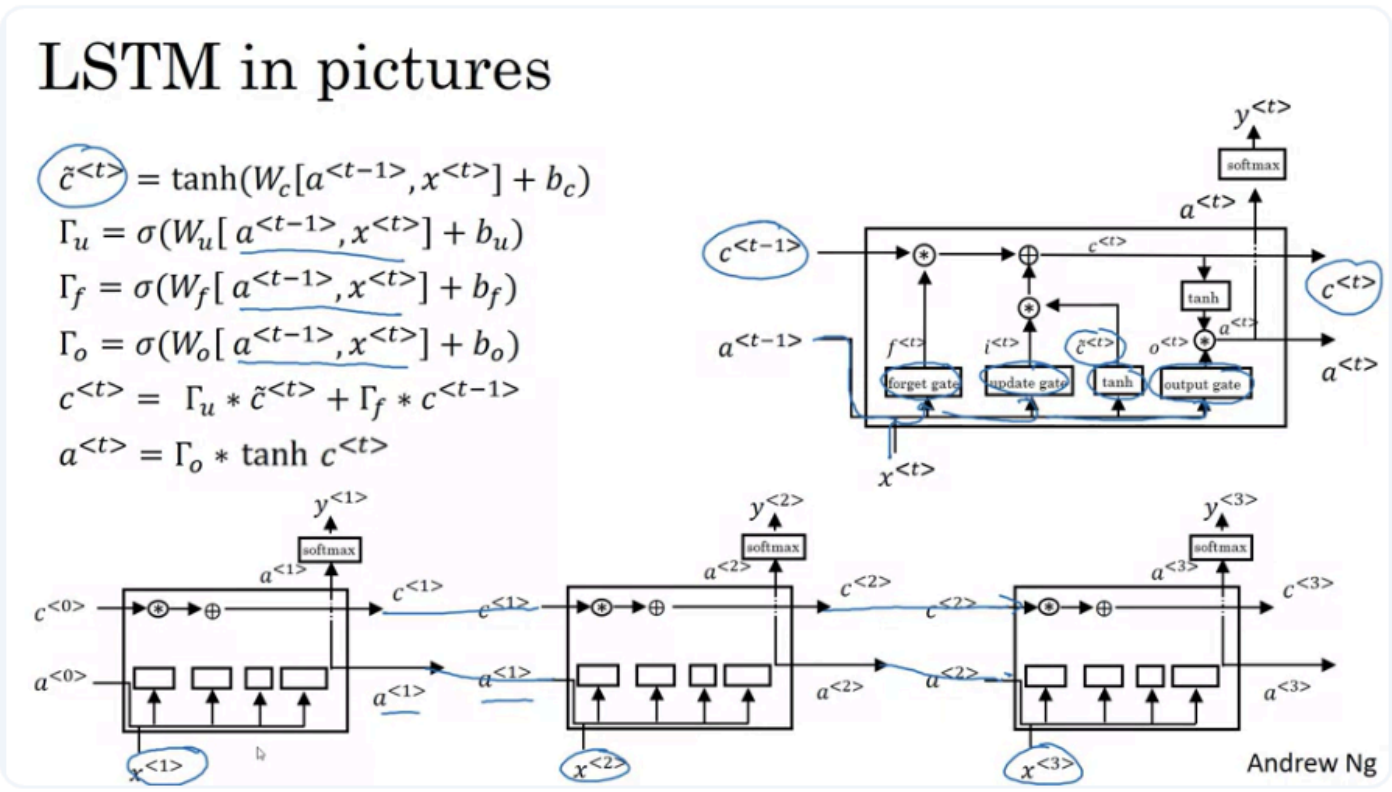
Useful topics to study:

- Input and output shapes
- Initial hidden states
- Multiple GRU layers
- Bidirectional GRUs
- Packed variable-length sequences

Long Short Term Memory (LSTM)

Long Short Term Memory





Long Short-Term Memory Networks

A **Long Short-Term Memory network**, or **LSTM**, is a gated recurrent architecture designed to learn long-range dependencies and reduce the vanishing gradient problem.

Like a GRU, an LSTM controls how information is stored, preserved, and removed over time. However, an LSTM is more flexible because it uses:

- A separate memory cell
- A separate hidden activation
- Three main gates

From GRU to LSTM

In the GRU discussed previously:

$a^t = C^t$

The hidden activation and memory cell are the same value.

The GRU also uses two gates:

- Update gate Γ_u
- Reset or relevance gate Γ_r

The LSTM differs because:

$a^t \neq C^t$

It maintains two separate states:

- C^t: memory cell state
- a^t: hidden activation or output state

This separation gives the LSTM more control over what it remembers internally and what it exposes as an output.

Main Components of an LSTM

At each time step, the LSTM receives:

- The current input x^t
- The previous hidden activation a^{t-1}
- The previous cell state C^{t-1}

It computes:

- Candidate cell value $\tilde{C}^t$
- Update gate Γ_u
- Forget gate Γf
- Output gate Γ_o
- New cell state C^t
- New hidden activation a^t

Candidate Cell Value

The LSTM first computes a candidate value that may be added to the memory cell:

$\tilde{C}^t = \tanh(W_c[a^{t-1}, x^t] + b_c)$

where:

- a^{t-1} is the previous hidden activation
- x^t is the current input
- W_c and b_c are learned parameters
- $\tilde{C}^t$ is the candidate memory content

The candidate contains new information that the model may choose to store.

Update Gate

The update gate determines how much of the candidate memory should be written into the cell state.

$\Gamma_u = \sigma(W_u[a^{t-1}, x^t] + b_u)$

The sigmoid function ensures:

$0 \leq \Gamma_u \leq 1$

For intuition:

- $\Gamma_u \approx 1$: write the candidate information
- $\Gamma_u \approx 0$: ignore the candidate information

Some literature calls this the **input gate** rather than the update gate.

Forget Gate

The forget gate determines how much of the previous cell state should be retained.

$\Gamma_f = \sigma(W_f[a^{t-1}, x^t] + b_f)$

For intuition:

- $\Gamma_f \approx 1$: preserve the old memory
- $\Gamma_f \approx 0$: forget the old memory

The forget gate allows the model to remove information that is no longer useful.

Cell-State Update

The new memory cell is computed as:

$C^t = \Gamma_u \odot \tilde{C}^t + \Gamma_f \odot C^{t-1}$

where $\odot$ represents element-wise multiplication.

This equation combines:

- New candidate information
- Previously stored information

Unlike the simplified GRU, the LSTM uses two independent gates:

- Γ_u controls the new candidate
- Γ_f controls the old cell state

This gives the LSTM more flexibility.

Comparison with the GRU Update

A simplified GRU uses:

$C^t = \Gamma_u \odot \tilde{C}^t + (1 - \Gamma_u) \odot C^{t-1}$

The same gate controls both terms.

If Γ_u increases:

- More new information is used
- Less old information is preserved

In an LSTM:

$C^t = \Gamma_u \odot \tilde{C}^t + \Gamma_f \odot C^{t-1}$

The update and forget decisions are independent.

Therefore, the LSTM can:

- Add new information while retaining old information
- Remove old information without adding anything new
- Replace old memory with new memory
- Preserve the memory without modifying it

Output Gate

The output gate determines which parts of the cell state should be exposed as the hidden activation.

$\Gamma_o = \sigma(W_o[a^{t-1}, x^t] + b_o)$

The final hidden activation is:

$a^t = \Gamma_o \odot \tanh(C^t)$

The lecture presents the core idea as the output gate controlling the value exposed from C^t.

The cell state may contain long-term information, while the hidden activation exposes only the parts currently needed by the rest of the network.

Complete LSTM Equations

Candidate Cell

$\tilde{C}^t = \tanh(W_c[a^{t-1}, x^t] + b_c)$

Update Gate

$\Gamma_u = \sigma(W_u[a^{t-1}, x^t] + b_u)$

Forget Gate

$\Gamma_f = \sigma(W_f[a^{t-1}, x^t] + b_f)$

Output Gate

$\Gamma_o = \sigma(W_o[a^{t-1}, x^t] + b_o)$

Cell-State Update

$C^t = \Gamma_u \odot \tilde{C}^t + \Gamma_f \odot C^{t-1}$

Hidden Activation

$a^t = \Gamma_o \odot \tanh(C^t)$

Intuition for the Three Gates

Update Gate

Answers:

> What new information should be written?

It controls how much of $\tilde{C}^t$ enters the memory cell.

Forget Gate

Answers:

> What old information should be removed?

It controls how much of C^{t-1} remains in the memory.

Output Gate

Answers:

> What information should be exposed now?

It controls which parts of the cell state contribute to a^t.

Example: Remembering Subject Number

Consider:

> The cats, which already ate a large amount of food, were full.

The LSTM may behave as follows.

When Reading “cats”

The candidate cell represents:

> subject number = plural

The update gate opens:

$\Gamma_u \approx 1$

The information is stored in C^t.

While Reading the Middle Clause

For words such as:

> which already ate a large amount of food

the forget gate remains open:

$\Gamma f \approx 1$

The update gate for the relevant memory dimension remains mostly closed:

$\Gamma_u \approx 0$

The plural information is preserved.

When Predicting “were”

The output gate exposes the stored subject-number information:

$\Gamma_o \approx 1$

The model can use it to choose the plural verb:

> were

After the Dependency Is Resolved

The model may close the forget gate for that memory dimension:

$\Gamma f \approx 0$

This removes the information when it is no longer needed.

Why LSTMs Help with Vanishing Gradients

The cell state has a relatively direct path through time:

$C^{t-1} \rightarrow C^t \rightarrow C^{t+1} \rightarrow \dots$

The important part of the update is:

$C^t = \Gamma f \odot C^{t-1} + \text{new information}$

If the forget gate is close to 1 and the update gate is close to 0:

$\Gamma f \approx 1$

$\Gamma_u \approx 0$

then:

$C^t \approx C^{t-1}$

The memory can remain almost unchanged across many time steps.

For example:

$C^3 \approx C^2 \approx C^1 \approx C^0$

This creates a more direct path for both:

- Information during forward propagation
- Gradients during backpropagation

As a result, the model can learn dependencies across much longer sequences than a basic RNN.

Separate Cell and Hidden States

The distinction between C^t and a^t is central to the LSTM.

Cell State C^t

Represents internal long-term memory.

It can carry information across many time steps, even when that information is not currently needed for an output.

Hidden Activation a^t

Represents the information exposed at the current time step.

It may be used to:

- Produce a prediction
- Pass information to the next LSTM unit
- Connect to another neural-network layer

The relationship is:

$a^t = \Gamma_o \odot \tanh(C^t)$

Therefore, the output gate can hide some stored information without deleting it from the memory cell.

Connecting LSTM Units Through Time

When LSTM units are connected sequentially, two values are passed forward:

- Hidden activation a^t
- Cell state C^t

At the next time step, the unit receives:

a^{t-1} and C^{t-1}

Conceptually:

$(x^1, a^0, C^0) \rightarrow \text{LSTM} \rightarrow (a^1, C^1)$

$(x^2, a^1, C^1) \rightarrow \text{LSTM} \rightarrow (a^2, C^2)$

$(x^3, a^2, C^2) \rightarrow \text{LSTM} \rightarrow (a^3, C^3)$

The cell-state path makes it possible for a stored value to travel across many time steps.

With suitable gate values:

$C^3 \approx C^2 \approx C^1 \approx C^0$

Vector-Valued Gates

The gates and states are usually vectors.

For a hidden size of 100:

$C^t \in \mathbb{R}^{100}$

$a^t \in \mathbb{R}^{100}$

$\Gamma_u \in \mathbb{R}^{100}$

$\Gamma_f \in \mathbb{R}^{100}$

$\Gamma_o \in \mathbb{R}^{100}$

$\tilde{C}^t \in \mathbb{R}^{100}$

Each gate controls every memory dimension independently.

For example:

$\Gamma_f = [1, 0, 0.7, \ldots]$

This may mean:

- Preserve the first memory component
- Completely forget the second
- Partially preserve the third

Different dimensions can store different types of information, such as:

- Subject number
- Sentence topic
- Sentiment
- Named entities
- Temporal context
- Syntactic structure

Element-Wise Cell Update Example

Suppose:

$\Gamma_u = [1, 0, 0.5]$

$\tilde{C}^t = [4, 8, 6]$

$\Gamma f = [0, 1, 0.8]$

$C^{t-1} = [2, 3, 10]$

Then:

$C^t = \Gamma_u \odot \tilde{C}^t + \Gamma f \odot C^{t-1}$

$C^t = [1\times4 + 0\times2, 0\times8 + 1\times3, 0.5\times6 + 0.8\times10]$

$C^t = [4, 3, 11]$

Therefore:

- The first component is replaced
- The second component is preserved
- The third combines old and new information

Peephole Connections

A common LSTM variation is the **peephole connection**.

In a standard LSTM, the gates depend on:

- a^{t-1}
- x^t

With peephole connections, the gates may also depend on the previous cell state:

C^{t-1}

For example:

$\Gamma f = \sigma(Wf[a^{t-1}, x^t] + Pf \odot C^{t-1} + bf)$

Similar terms may be added to:

- The update gate
- The forget gate
- The output gate

This allows the gates to inspect the memory cell directly when deciding whether to update, forget, or expose information.

One-to-One Peephole Relationships

For a 100-dimensional cell state, peephole connections are often element-wise.

The first element of C^{t-1} affects the first element of the gate.

The second element affects the second element, and so on.

Conceptually:

$C^{t-1}[i] \rightarrow gate[i]$

rather than:

$C^{t-1}[i]$ → every gate dimension

This is often implemented using element-wise multiplication with a learned parameter vector.

Simplified LSTM Pseudocode

```
candidate = tanh(
    W_candidate @ concat(previous_hidden, current_input)
    + b_candidate
)

update_gate = sigmoid(
    W_update @ concat(previous_hidden, current_input)
    + b_update
)

forget_gate = sigmoid(
    W_forget @ concat(previous_hidden, current_input)
    + b_forget
)

output_gate = sigmoid(
    W_output @ concat(previous_hidden, current_input)
    + b_output
)

current_cell = (
    update_gate * candidate
    + forget_gate * previous_cell
)

current_hidden = output_gate * tanh(current_cell)
```

All multiplications between gates and states are element-wise.

LSTM vs. Basic RNN

Aspect	Basic RNN	LSTM
Main state	Hidden state	Cell state and hidden state
Gates	None	Update, forget, and output
Memory control	Implicit	Explicit
Long-term dependencies	Difficult	Much stronger
Vanishing gradients	Common	Reduced
Parameters	Fewer	More
Computation	Faster	More expensive
Architecture	Simple	More complex

LSTM vs. GRU

Aspect	GRU	LSTM
Main states	One combined state	Cell state and hidden state
Number of main gates	Two	Three
Gates	Update and reset	Update, forget, and output
Candidate calculation	Uses reset gate	Usually no reset gate
Complexity	Simpler	More complex

Aspect	GRU	LSTM
Parameter count	Lower	Higher
Computation	Usually faster	Usually slower
Flexibility	High	Higher
Historical use	Newer	Older and well established

When to Use a GRU

A GRU may be preferred when:

- Training speed matters
- Computational resources are limited
- The dataset is relatively small
- A simpler architecture is desirable
- You want to build a larger network with fewer parameters

Because it has fewer gates, a GRU is often faster and easier to scale.

When to Use an LSTM

An LSTM may be preferred when:

- Long-term memory is especially important
- More flexible control over memory is useful
- The dataset and computing resources are sufficient
- You want a historically well-tested recurrent architecture

The lecture describes LSTM as a strong default choice because it has been used successfully across many sequence-modeling problems.

Is LSTM Always Better Than GRU?

No architecture is universally superior.

Depending on the task:

- An LSTM may perform better
- A GRU may perform equally well
- A GRU may train faster
- A simpler model may generalize better

The appropriate choice should be determined experimentally.

Useful comparison criteria include:

- Validation loss
- Accuracy
- Training speed
- Memory usage
- Parameter count
- Long-range dependency performance

Historical Context

LSTMs were introduced before GRUs.

The LSTM architecture is associated with:

- Sepp Hochreiter

- Jürgen Schmidhuber

The original LSTM work had a major impact on sequence modeling and included substantial analysis of the vanishing-gradient problem.

GRUs were introduced later as a simpler gated recurrent architecture.

Key Takeaways

- LSTM stands for Long Short-Term Memory.
- It is designed to capture long-range dependencies.
- Unlike a GRU, an LSTM has separate cell and hidden states.
- The cell state C^t stores long-term information.
- The hidden activation a^t exposes information needed at the current step.
- The update gate controls what new information is written.
- The forget gate controls what old information is preserved.
- The output gate controls what information is exposed.
- The cell-state update uses separate update and forget gates.
- A nearly unchanged cell-state path helps reduce vanishing gradients.
- Peephole connections allow gates to inspect the previous cell state.
- LSTMs are more flexible but more computationally expensive than GRUs.
- GRUs are simpler and often faster.
- Neither architecture is best for every task.

Further Learning

Understanding LSTM Networks

Study visual explanations of:

- Cell-state flow
- Update or input gate
- Forget gate
- Output gate
- Candidate memory

A widely referenced resource is Christopher Olah's article:

> Understanding LSTM Networks

LSTM Variants

Explore common variations such as:

- Peephole LSTM
- Coupled input–forget gates
- Bidirectional LSTM
- Stacked LSTM
- Convolutional LSTM
- Projection LSTM

Practical Comparison

Train a basic RNN, GRU, and LSTM on the same task and compare:

- Parameter count

- Epoch duration
- Final loss
- Gradient behavior
- Performance on long sequences

PyTorch LSTM

A basic LSTM layer can be defined as:

```
import torch.nn as nn

lstm = nn.LSTM(
    input_size=input_dim,
    hidden_size=hidden_dim,
    num_layers=1,
    batch_first=True,
)
```

Unlike a GRU, the initial state contains two tensors:

- Initial hidden state h^0
- Initial cell state C^0

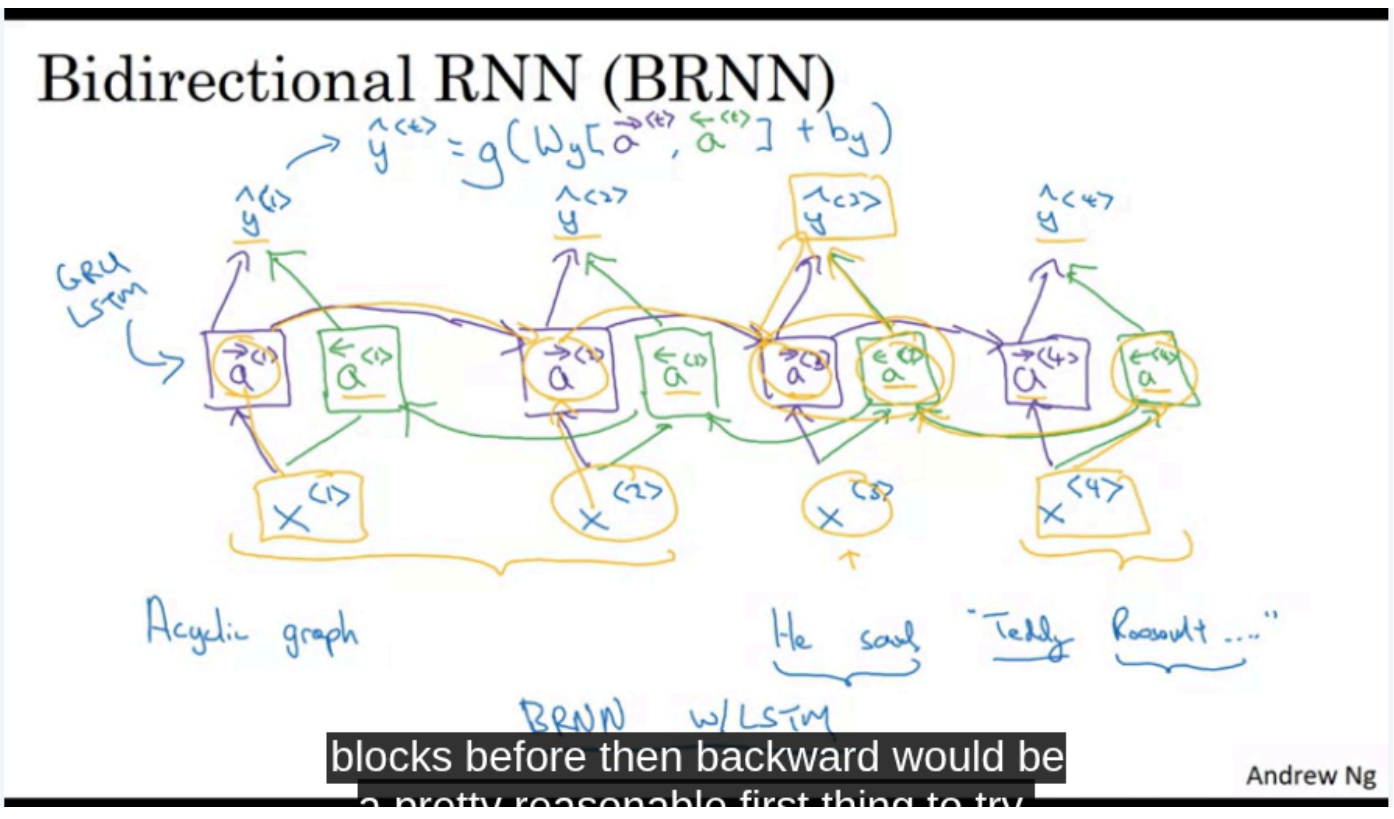
Next Topics

Useful continuations include:

- Bidirectional RNNs
- Deep recurrent neural networks
- Sequence-to-sequence architectures
- Attention mechanisms
- Transformer models

We could read this: link for more detail

Bidirectional RNN



Bidirectional Recurrent Neural Networks

A **Bidirectional Recurrent Neural Network**, or **BRNN**, is an RNN architecture that uses information from both:

- Earlier positions in a sequence
- Later positions in a sequence

This allows the prediction at each time step to depend on the entire input sequence rather than only the previous inputs.

Limitation of a Unidirectional RNN

A standard RNN processes a sequence from left to right:

$x^{1} \rightarrow x^{2} \rightarrow x^{3} \rightarrow \ldots \rightarrow x^{T}$

At time step t, its prediction depends only on:

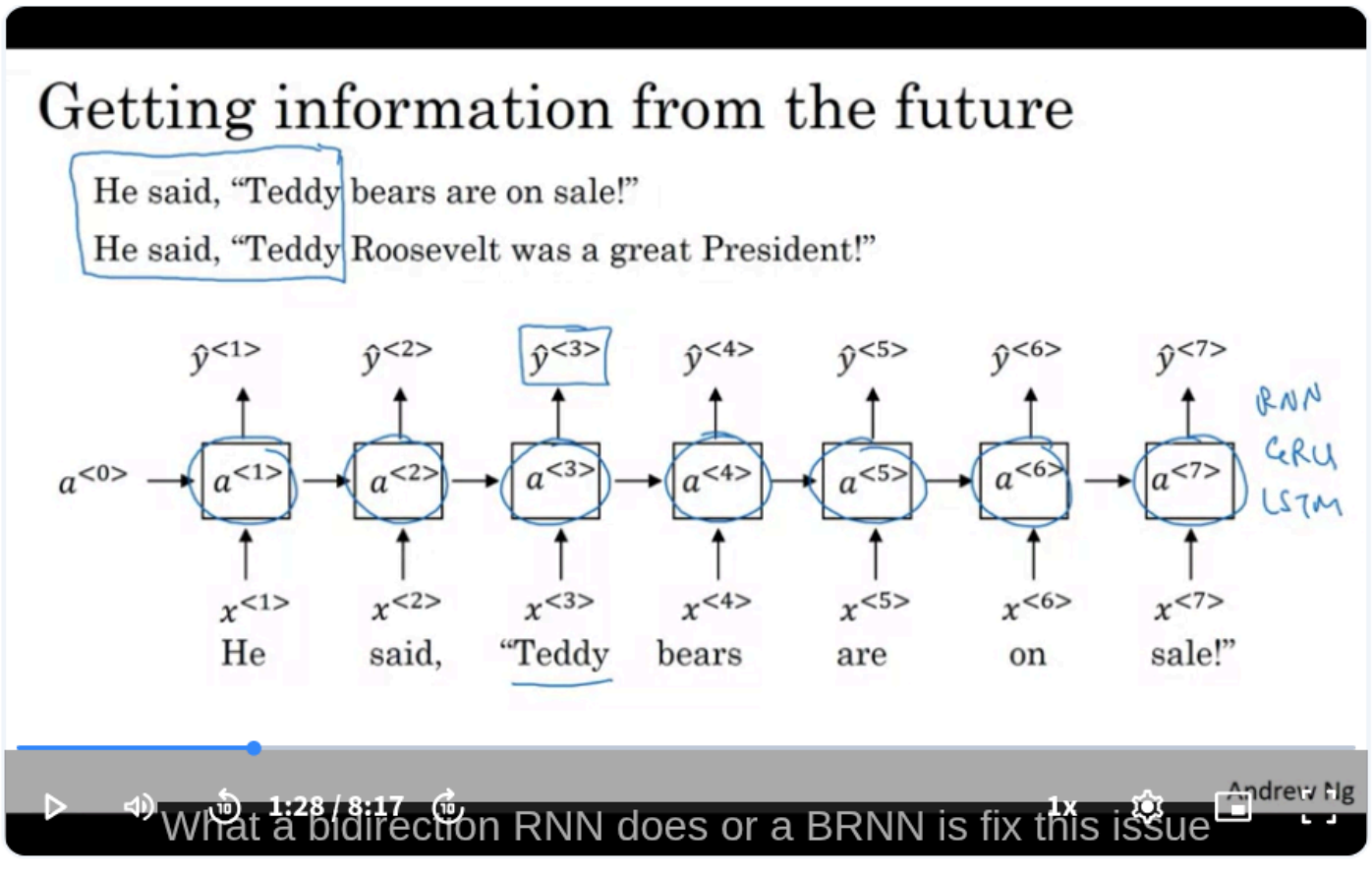
$x^{1}, x^{2}, \ldots, x^{t}$

It cannot use information from future positions:

$x^{t+1}, x^{t+2}, \ldots, x^{T}$

This can be a problem when later words are needed to interpret an earlier word.

Motivation: Named Entity Recognition



Consider the partial phrase:

> He said Teddy ...

Based only on these words, it is unclear whether **Teddy** refers to:

- A person
- A teddy bear
- Another entity

The following word may resolve the ambiguity:

> He said Teddy Roosevelt ...

Now it is clear that **Teddy** is part of a person's name.

To correctly label **Teddy**, the model needs information from both:

- The preceding context: "He said"
- The following context: "Roosevelt"

A forward-only RNN cannot use "Roosevelt" when predicting the label for "Teddy."

Basic BRNN Architecture

A bidirectional RNN contains two separate recurrent networks:

1. A forward recurrent network
2. A backward recurrent network

Both networks process the same input sequence but in opposite directions.

For an input sequence:

x^1, x^2, x^3, x^4

the forward network processes:

$x^1 \rightarrow x^2 \rightarrow x^3 \rightarrow x^4$

The backward network processes:

$x^4 \rightarrow x^3 \rightarrow x^2 \rightarrow x^1$

Forward Hidden States

The forward hidden states are written as:

$\vec{a}^1, \vec{a}^2, \vec{a}^3, \vec{a}^4$

The right arrow indicates the forward direction.

The forward recurrence is:

$\vec{a}^t = g(W_{\vec{x}} x^t + W_{\vec{a}} \vec{a}^{t-1} + \vec{b})$

At time step t, $\vec{a}^t$ summarizes information from:

$x^1, x^2, \ldots, x^t$

For example:

$\vec{a}^3$ contains information from x^1, x^2, and x^3.

Backward Hidden States

The backward hidden states are written as:

$\overleftarrow{a}^1, \overleftarrow{a}^2, \overleftarrow{a}^3, \overleftarrow{a}^4$

The left arrow indicates the backward direction.

The backward recurrence is:

$\overleftarrow{a}^t = g(W_{\overleftarrow{x}} x^t + W_{\overleftarrow{a}} \overleftarrow{a}^{t+1} + \overleftarrow{b})$

At time step t, $\overleftarrow{a}^t$ summarizes information from:

$x^t, x^{t+1}, \ldots, x^T$

For example:

$\overleftarrow{a}^3$ contains information from x^3 and x^4.

Computation Order

The forward and backward networks are computed separately.

Forward Pass

Compute from left to right:

$\vec{a}^1 \rightarrow \vec{a}^2 \rightarrow \vec{a}^3 \rightarrow \vec{a}^4$

Backward Pass

Compute from right to left:

$\overleftarrow{a}^4 \rightarrow \overleftarrow{a}^3 \rightarrow \overleftarrow{a}^2 \rightarrow \overleftarrow{a}^1$

After both directions have been computed, their hidden states are combined to produce the output at each position.

Output Prediction

At time step t, the prediction uses both hidden states:

$\hat{y}^t = g_Y(W_Y[\vec{a}^t, \overleftarrow{a}^t] + b_Y)$

where:

- $\vec{a}^t$ contains information from the past and present
- $\overleftarrow{a}^t$ contains information from the future and present
- $[\vec{a}^t, \overleftarrow{a}^t]$ represents concatenation
- $\hat{y}^t$ is the prediction at time step t

The model therefore uses context from the entire sequence.

Information Available at Each Position

For a sequence:

x^1, x^2, x^3, x^4

the prediction at time step 3 uses:

- x^1 through the forward network
- x^2 through the forward network
- x^3 through both networks
- x^4 through the backward network

Therefore:

$\hat{y}^3$ depends on x^1, x^2, x^3, and x^4

More generally:

$\hat{y}^t$ can depend on $x^1, x^2, \ldots, x^T$

This is the main advantage of bidirectional recurrence.

Example: Labeling “Teddy”

Consider:

> He said Teddy Roosevelt was a great president.

To predict whether **Teddy** is part of a person’s name:

Forward Context

The forward state may contain:

> He said Teddy

This alone is ambiguous.

Backward Context

The backward state contains information from:

> Teddy Roosevelt was a great president

The presence of **Roosevelt** strongly suggests that **Teddy** is a person’s name.

By combining both states, the BRNN can make a more accurate prediction.

BRNN Building Blocks

The forward and backward recurrent units do not have to be basic RNN cells.

A BRNN can use:

- Basic RNN units
- GRU units
- LSTM units

Common architectures include:

- Bidirectional RNN
- Bidirectional GRU
- Bidirectional LSTM

A **Bidirectional LSTM**, often called **BiLSTM**, has historically been a common choice for many NLP sequence-labeling tasks.

Bidirectional LSTM

In a BiLSTM:

- One LSTM processes the sentence from left to right
- Another LSTM processes it from right to left
- Their outputs are combined at every token

For each time step:

$h^t = [\vec{h}^t, \overleftarrow{h}^t]$

The combined representation may then be used for:

- Named entity recognition
- Part-of-speech tagging
- Token classification
- Sequence labeling

Dimensions of a BRNN

Suppose each directional hidden state has dimension d.

Then:

$\vec{a}^t \in \mathbb{R}^d$

$\overleftarrow{a}^t \in \mathbb{R}^d$

After concatenation:

$[\vec{a}^t, \overleftarrow{a}^t] \in \mathbb{R}^{2d}$

Therefore, a bidirectional model usually produces representations twice as large as those of one directional network, unless an additional projection is applied.

Advantages of BRNNs

Access to Full Context

Each prediction can use information from the entire sequence.

This is helpful when the meaning of a token depends on words that appear later.

Better Token Representations

The representation at each position captures:

- Left context
- Current input
- Right context

This generally provides richer contextual information than a forward-only RNN.

Strong Performance on Sequence Labeling

BRNNs are effective for tasks where the complete input is available before predictions are needed.

Examples include:

- Named entity recognition
- Part-of-speech tagging
- Text classification
- Handwriting recognition
- Offline speech recognition
- Protein sequence analysis

Main Disadvantage

A standard BRNN needs the complete input sequence before it can produce fully informed predictions.

The backward network must begin at the final input:

x^{T}

Therefore, the model cannot compute the backward states until the end of the sequence is known.

Limitation for Real-Time Applications

Consider live speech recognition.

A person is speaking continuously:

> Today I would like to discuss ...

A forward RNN can begin processing immediately because it only needs previous audio frames.

A standard BRNN cannot fully process the current frame using backward context until it receives future frames.

In the most straightforward implementation, the system may need to wait until the speaker finishes the entire utterance.

This creates latency and makes the model less suitable for strict real-time applications.

Offline vs. Online Processing

Offline Processing

The entire input sequence is available before prediction.

Examples:

- Processing a complete sentence
- Transcribing a recorded audio file
- Analyzing a saved document
- Labeling a complete biological sequence

BRNNs are highly suitable for these settings.

Online Processing

The input arrives gradually, and predictions must be produced immediately.

Examples:

- Live speech recognition
- Real-time captioning
- Streaming sensor analysis
- Real-time anomaly detection

A standard BRNN is less suitable because future inputs are unavailable.

Alternatives for Streaming Tasks

For real-time applications, possible approaches include:

- Unidirectional RNNs
- Unidirectional GRUs or LSTMs
- Limited look-ahead windows
- Chunk-based bidirectional processing
- Latency-controlled bidirectional models
- Streaming attention architectures

These approaches trade some future context for lower latency.

BRNN vs. Unidirectional RNN

Aspect	Unidirectional RNN	Bidirectional RNN
Processing direction	Left to right	Left to right and right to left
Past context	Yes	Yes
Future context	No	Yes
Complete sequence required	No	Usually yes
Real-time suitability	Better	More difficult
Contextual representation	More limited	Richer
Computation	Lower	Higher
Common use	Generation and streaming	Offline sequence labeling

BRNNs and Language Generation

Bidirectional models are not normally used directly for left-to-right autoregressive text generation.

When generating the next token, future tokens do not yet exist.

For next-token prediction:

$P(y^t \mid y^1, \ldots, y^{t-1})$

the model must use only previous tokens.

Therefore, unidirectional architectures are more appropriate for:

- Language modeling
- Autoregressive text generation
- Real-time sequence generation

BRNNs are more appropriate when the full input sequence already exists and the goal is to analyze or label it.

BRNNs and Named Entity Recognition

Named entity recognition assigns a label to every token.

Example:

Token	Label
Teddy	Person
Roosevelt	Person
was	Other
president	Other

A bidirectional model improves these predictions because each label can depend on the complete sentence.

The output sequence usually has the same length as the input sequence:

$T_x = T_y$

This makes BRNNs well suited to many-to-many sequence-labeling problems.

Forward and Backward Parameters

The forward and backward networks usually have separate parameters.

Forward parameters:

$W_{\vec{a}}$, $W_{\vec{x}}$, $\vec{b}$

Backward parameters:

$W_{\overleftarrow{a}}$, $W_{\overleftarrow{x}}$, $\overleftarrow{b}$

They do not normally share weights because the two directions may learn different patterns.

For example:

- The forward network learns how earlier words influence later words.
- The backward network learns how later words help interpret earlier words.

Training a BRNN

The complete model is trained using backpropagation.

The process includes:

1. Compute all forward hidden states.
2. Compute all backward hidden states.
3. Combine the states at each position.
4. Calculate the predictions.
5. Compute the sequence loss.
6. Backpropagate through both recurrent directions.
7. Update all forward, backward, and output parameters.

For token-level classification, the total loss may be:

$L = \Sigma_{t=1}^{T} L(\hat{y}^t, y^t)$

Simplified Pseudocode

```
forward_states = forward_rnn(inputs)

backward_states_reversed = backward_rnn(
    reverse(inputs)
)

backward_states = reverse(
    backward_states_reversed
)

outputs = []

for forward_state, backward_state in zip(
    forward_states,
    backward_states,
):
    combined_state = concatenate(
        forward_state,
        backward_state,
    )

    prediction = output_layer(combined_state)
    outputs.append(prediction)
```

The second reversal aligns the backward states with their original token positions.

PyTorch BiLSTM Example

```
import torch.nn as nn

bilstm = nn.LSTM(
    input_size=input_dim,
    hidden_size=hidden_dim,
    num_layers=1,
    batch_first=True,
    bidirectional=True,
)
```

With:

`bidirectional=True`

PyTorch creates one forward LSTM and one backward LSTM.

If each direction has hidden size H, the output dimension is:

2H

Example:

```
outputs, (hidden, cell) = bilstm(inputs)

print(outputs.shape)
# (batch_size, sequence_length, 2 * hidden_dim)
```

Key Equations

Forward State

$\vec{a}^{t} = g(W_{\vec{x}}x^{t} + W_{\vec{a}}\vec{a}^{t-1} + \vec{b})$

Backward State

$\overleftarrow{a}^{t} = g(W_{\overleftarrow{x}}x^{t} + W_{\overleftarrow{a}}\overleftarrow{a}^{t+1} + \overleftarrow{b})$

Combined Representation

$a^{t} = [\vec{a}^{t}, \overleftarrow{a}^{t}]$

Output

$\hat{y}^{t} = g_{Y}(W_{Y}a^{t} + b_{Y})$

Key Takeaways

- A bidirectional RNN processes the input in both directions.
- The forward network summarizes earlier and current inputs.
- The backward network summarizes later and current inputs.
- Their hidden states are combined at each time step.
- A BRNN allows predictions to use information from the entire sequence.
- This is useful when later context clarifies an earlier token.
- BRNNs can use basic RNN, GRU, or LSTM cells.
- BiLSTMs have been widely used for NLP sequence-labeling tasks.
- A standard BRNN requires the complete input sequence.

- This makes it effective for offline processing but less suitable for strict real-time applications.
- BRNNs are commonly used for analysis and labeling, not autoregressive generation.
- The next architectural extension is to stack recurrent layers to create a deep RNN.

Further Learning

Deep Recurrent Neural Networks

The next topic is how multiple recurrent layers can be stacked.

Important ideas include:

- Vertical connections between recurrent layers
- Recurrent connections through time
- Hidden states indexed by layer and time
- Computational cost of deep recurrent networks

Bidirectional LSTM for NER

A useful practical architecture is:

Input embeddings → BiLSTM → Linear layer → Token labels

An additional Conditional Random Field layer may be used to model dependencies between neighboring output labels:

Input embeddings → BiLSTM → CRF

Contextual Sequence Models

Compare BRNNs with later contextual architectures:

- ELMo
- BERT
- Bidirectional Transformers
- Encoder-only Transformer models

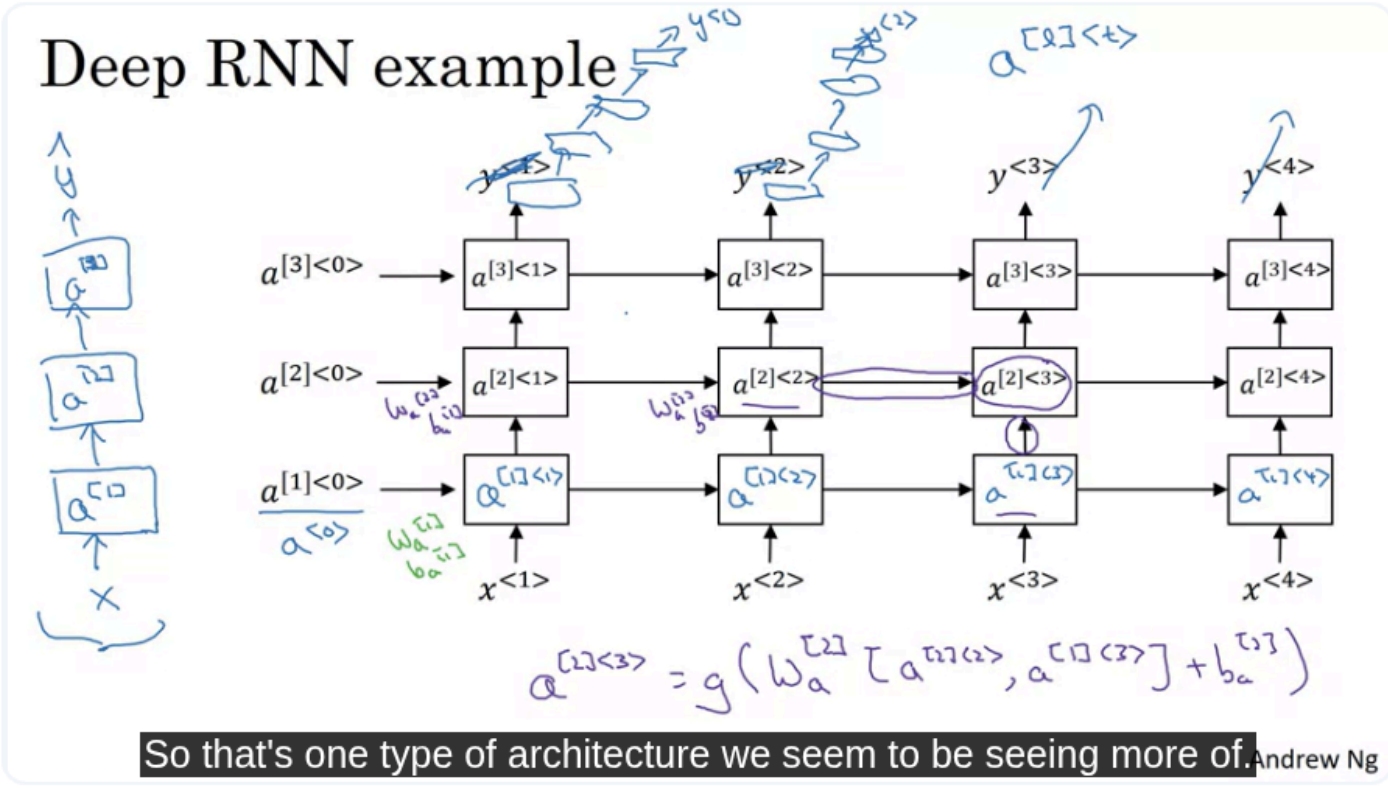
Like BRNNs, these models use context from both sides of a token when the full input is available.

Streaming Bidirectional Models

For speech and online processing, explore:

- Chunked BiLSTM
- Look-ahead context
- Latency-controlled BiLSTM
- Streaming Transformer encoders
- Blockwise processing

Deep RNNs



Deep RNNs

Save note